Vision X Multi Modal Deepfake Detection

B. Tech

B Tech project Work in

Department of Computer Science and Engineering

Submitted By :

Student Name	Reg. No.	Course Code:
Rishi Viswanatha Subramani	22ETMC412013	CSP402A
Anish Chhetri	22ETCS002011	CSP402A
Elaizah Foning Ramsong	22ETCS002035	CSP402A
Sharan Mathan Mattachotil	22ETCS002098	CSP402A

Supervisor: Dr. C. Narendra Babu

June – 2026

FACULTY OF ENGINEERING AND TECHNOLOGY

M. S. RAMAIAH UNIVERSITY OF APPLIED SCIENCES

Bengaluru -560054

FACULTY OF ENGINEERING AND TECHNOLOGY

RAMAIAH UNIVERSITY OF APPLIED SCIENCES

Certificate

This is to certify that the Project Work titled "Vision X Multi Modal Deepfake Detection" is a bonafide record of the work carried out by Mr. Rishi Viswanatha Subramani, Reg. No. 22ETMC412013, Mr. Anish Chhetri, Reg. No. 22ETCS002011, Mr. Elaizah Foning Ramsong, Reg. No. 22ETCS002035, Mr. Sharan Mathan Mattachotil, Reg. No. 22ETCS002098 in partial fulfilment of requirements for the award of B.Tech Degree of M. S. Ramaiah University of Applied Sciences in the Department of Computer Science & Engineering

June 2026

Dr. C. Narendra Babu
Supervisor

Dr. C. Narendra Babu
Head - Department of CSE,FET

Dr. Sarat Kumar Maharana
Dean, FET

Examiners

Name of Examiners	**Signature with date**
1.	
2.	

Declaration

Vision X: Multi-Modal Deepfake Detection

The project work is submitted in partial fulfilment of academic requirements for the B.Tech. Degree of M. S. Ramaiah University of Applied Sciences in the Department of CSE. This project work is a result of our own work and is in conformance to the guidelines on plagiarism as laid out in the University Student Handbook. All sections of the text and results, which have been obtained from other sources are fully referenced. We understand that cheating and plagiarism constitute a breach of University regulations, hence this project report has been passed through plagiarism check and the report has been submitted to the supervisor.

Sl. No.	Reg. No.	Student Name	Signature
1.	22ETMC412013	Rishi Viswanatha Subramani	
2.	22ETCS002011	Anish Chhetri	
3.	22ETCS002035	Elaizah Foning Ramsong	
4.	22ETCS002098	Sharan Mathan Mattachotil	

Date : 5 June 2026

Acknowledgements

We take this opportunity to express our sincere gratitude and appreciation to **Ramaiah University of Applied Sciences, Bengaluru** for providing us an opportunity to carry out our group project work.

It brings a sense of privilege to associate this work **Dr. Narendra Babu Chindanur, Head, Department of CSE.** Working under him has been a precious opportunity.

We are immensely grateful for their vigilant supervision, constant encouragement, inspiring ideas, constructive criticism, and timely expertise, which reflects in the final images of this work.

We are grateful to the University and thank **Dr. Narendra Babu Chindanur, Head, Department CSE,** for providing us with this opportunity, guidance, and encouragement throughout.

We would like to earnestly thank **Dr. Sarat Kumar Maharana, Dean, RTC,** for facilitating academic excellence in the college that helped in completing this project.

We would like to earnestly thank **Prof. Kuldeep Kumar Raina, Vice Chancellor, RUAS,** for facilitating academic excellence in the college that helped in completing this project.

We are thankful to the management of **Ramaiah University of Applied Sciences** for providing all the facilities and resources for the successful completion of the course.

Finally, we thank our parents, friends and all our well-wishers who have supported us in this endeavor.

The values of dedication, discipline, work ethics and patience that we have imbibed here will always remain the guiding force throughout our lives.

Abstract

With increasing advancements in technology, AI manipulation of faces in media, referred to as deepfakes, is an increasingly concerning threat to digital trust, personal privacy, and information integrity. Existing cloud-based detection tools pose privacy risks for individuals and organisations as they require users to upload sensitive data to remote servers. VisionX aims to address this critical gap as a privacy focused, edge-deployed, multi-modal deepfake detection platform. All inference is executed locally on the user's machine, ensuring that no data ever leaves the device.

VisionX conducts deepfake detection for three media types: images, audio, and video.
For image and video analysis, the platform employs a two-stage pipeline combining MediaPipe's facial landmark geometry extraction with a locally running Qwen3 VLM, augmented by a forensic pipeline containing 4 different methods of analysis for further verification.For audio, the system employs a MFCC based pre-processing pipeline with a LAION-CLAP model being used. Furthermore, the implementation of a Video Watermarking system allows the user to embed a digital signature onto manipulated videos.

After conducting testing on various real world use cases, the system was able to deliver precise and reliable deepfake detection for different manipulated image, video and audio media files. Further research and training on more extensive datasets will increase the performance capability even more. The inclusion of the watermarking system adds more versatility to the system. This system demonstrates that high-accuracy, reliable, multi-signal deepfake detection can be delivered entirely on the edge locally without any dependency on cloud infrastructure.

Table of Contents

List of Tables

List of Figures

Nomenclature

f	Frequency (Hz)
P	Probability score [0 – 1]
px	Pixel
SR	Sampling Rate (Hz)
T	Time (seconds)
w, h	Image width and height (pixels)

Abbreviation and Acronyms

API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
CFA	Camera Fingerprint Analysis
CNN	Convolutional Neural Network
CPU	Central Processing Unit
ELA	Error Level Analysis
EXIF	Exchangeable Image File Format
FFT	Fast Fourier Transform
GAN	Generative Adversarial Network
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
MFCC	Mel-Frequency Cepstral Coefficient
ML	Machine Learning
ONNX	Open Neural Network Exchange
REST	Representational State Transfer
UI	User Interface
VLM	Vision Language Model
CLAP	Contrastive Language-Audio Pretraining

Self-Assessment of the Project: Vision-X Multi Modal Deepfake Detection

Levels: Poor-1, Good-2, Excellent-3			
	PO PSO	**Contribution from the project**	**Level**
1	**Engineering Knowledge:** Knowledge of mathematics, engineering fundamentals engineering specialization to form of complex engineering problems	Applying vision-language models (Qwen3-VL-4B), CNNs, and signal processing techniques (FFT, MFCC) to for deepfake detection as well as cryptography techniques like Shamir's Secret Sharing for Video Watermarking	3
2	**Problem Analysis:** Identify, formulate, review, research literature & analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.	Model deepfake detection across image, audio, and video formats to automatically identify and classify complex data manipulations in images, audio and videos.	3
3	**Design/development of solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and	Design a privacy-first, edge-deployed platform with a FastAPI backend and React frontend for detecting deepfakes entirely on the user's local machine.	3

	safety, whole-life cost, net zero carbon, culture, society and environment as required		
4	**Conduct investigations of complex problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.	Conduct investigations using multi-signal digital forensics, including EXIF, Error Level Analysis (ELA), and Camera Fingerprint Analysis (CFA) to differentiate authentic media from AI-generated content.	3
5	**Modern tool usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction & modelling recognizing their limitations to solve complex engineering problems.	Utilizing modern tools including FastAPI, React 19, ONNX Runtime, and llama-cpp-python for local machine learning edge inference.	3
6	**The Engineer and the world:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.	Protect personal privacy and information integrity by ensuring all sensitive media is processed locally without uploading to cloud servers.	3
7	**Ethics:** Apply ethical principles and commit to professional ethics, human	Project work and report followed honor code which is verified by	3

	values, diversity and inclusion; adhere to national & international laws.	Plagiarism check (25%) and report conforming to Industry standard.	
8	**Individual and Team Work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.	Active participation was experienced among all the team members despite having to overcome many challenges.	3
9	**Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences	Effective documentation is done using Latex (Overleaf) and presented using structure easy to understand. Effective presentation is also prepared highlighting the novelty, design solution, result analysis and inference giving directions for further improvement.	3
10	**Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.	Scheduling and plan of action was prepared at the beginning. Execution was done using Cost efficient, scalable and customization. approach.	2
11	**Life-long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and	Deepfake generation techniques (such as GANs and diffusion models) evolve rapidly; the project builds a modular architecture to adapt to new	3

	emerging technologies and iii) critical thinking in the broadest context of technological change	Generative AI models and forgery techniques.	
13	**PSO1:** Provide students with a strong foundation in mathematics and computing along with breadth and foundational requirement in computing, science, engineering and humanities to enable them to devise and deliver efficient and safe solutions to challenging problems in Computer Science and inter-disciplinary areas.	Applied machine learning algorithms, fine tuning ONNX models, and data pre-processing (such as MFCC extraction) to help identify AI generated content.	3
14	**PSO2:** Impart analytic and cognitive skills required to develop innovative solutions for R&D, to build creative, dependable and safe products for Industry based on dynamic societal requirements motivated and nurtured by sound theoretical and practical knowledge of time tested and long lasting principles of computer science, current tools and technologies.	Implemented an edge-deployed analytical platform using Qwen3-VL and CLAP models to solve data privacy and security challenges.	3
15	**PSO3:** Develop managerial and entrepreneurial skills inculcating strong human values along with social, interpersonal and leadership skills required for professional success in	Designed a user-friendly platform featuring an interactive AI assistant (Xite) and forensic dashboards to aid non-expert users in detection.	3

	evolving global professional environments		

SUSTAINABLE DEVELOPMENT GOALS (SDG) addressed in the project:

Levels: Poor:1, Good :2, Excellent:3		
SDG	**Level**	**Justification**
Good Health and Well-being	2	Promotes trust and enables public access to Information by providing tools to verify the authenticity of digital media and combat misinformation.
Quality education	2	Provides awareness about the danger of AI
Decent work and Economic Growth		
Industry, Innovation and Infrastructure	3	Enhancing digital infrastructure and combining technological capabilities in the fields of cybersecurity and AI.
Climate action		

Note: Select the SDG addressed and mark the level addressed in the project

Introduction

With the increasingly common spread of AI generated media, it is important to classify and detection of these media from human generated ones. Advancements in deep learning, including Generative Adversarial Networks (GANs) and diffusion models have made it simple and easy to generate realistic human faces, clone voices, and fabricate video content that is indistinguishable from authentic media. The real-world dangers of this technology include privacy violations and identity fraud as well as uncertainty in the integrity of data and spread of misinformation.

This report presents VisionX, a privacy-first, edge-deployed, multi-modal deepfake detection platform where the project aims to address both the challenges of detection accuracy and user privacy by running all inference locally on the user's device, eliminating the need for cloud-based processing of sensitive media.

1.1 Background and Motivation

Deepfake technology is built mainly on neural network architectures such as autoencoders and GANs, which learn to map a source identity onto a target identity with high photorealistic fidelity. Since initial release of the original deepfake tools circa 2017, the access to such tools has increased drastically. Today, consumer-grade hardware is sufficient for producing convincing synthetic media within minutes.

The consequences are wide-ranging. In the domain of personal security, deepfakes have been used to fabricate non-consensual imagery for defamation. In political contexts, synthetic video and audio of public figures have been used to spread disinformation. In finance, voice-cloning attacks have been used to impersonate executives and authorise fraudulent transactions. Legal frameworks are struggling to stay up to date with the technology, placing the burden of detection on the public and on platform operators.

Most existing detection tools operate as cloud services, users upload their media to a remote server, receive a verdict, and trust that the service handles their data

appropriately. This model poses severe risks with scenarios involving sensitive or personal media. VisionX was conceived to close this gap as a fully local, multi-signal detection system that analyses images, audio, and video on the user's own hardware, without any data leaving the device.

1.2 Overview of VisionX

VisionX is a web-based application consisting of a Python FastAPI backend and a React 19 frontend, communicating over a local HTTP interface. All machine learning inference is performed locally: image classification via llama-cpp-python running a 4-bit quantised Qwen3-VL-4B vision-language model, and audio classification via ONNX Runtime running a CLAP model.

The platform supports three modes of analysis alongside a video watermarking feature:

- **Image Mode:** Accepts JPEG and PNG uploads. Detects faces using MediaPipe's 468-point facial landmark model, classifies each detected face using the Qwen3-VL-4B VLM, and also runs a four-signal forensic pipeline. A composite verdict is returned for each image. An interactive AI assistant (Xite) is embedded in the image mode, allowing users to question the VLM's decision making in natural language.
- **Audio Mode:** Audio files (WAV/MP3) are first normalised, resampled to 16 kHz, segmented into three-second overlapping chunks, and the MFCC features are extracted before classification using the LAION-CLAP ONNX model.
- **Video Mode:** Accepts .mov .mp4 file type uploads. Utilises the aforementioned Qwen VLM model to carry out forensic analysis with reasoning and returns a verdict for the file’s authenticity.
- **Watermarking**: Accepts any video input, along with tags such as name of creator or model that generated the video, encrypts and divides the tag into several shares using Shamir Secret Sharing and embeds these shares in the pixel data of individual

frames. These can only be removed if the exact location of the modified pixel in each frame is known, so it prevents tampering and is resilient to compression.

1.3 Scope of the Project

The scope of VisionX as addressed in this report encompasses the following:

- Design and implementation of a fully local deepfake detection pipeline for static images.
- Development of a multi-signal forensic analysis engine combining VLM-based classification with signal-processing-based anomaly detection.
- Construction of an audio deepfake detection pipeline with using a LAION-CLAP model in an ONNX classification.
- Development of a React-based frontend providing an intuitive user interface for all three modes.
- Creation of a system to add a digital signature that is resilient against manipulation.
- A robust system architecture allowing modular expansion to new media types or AI models without having to restructure the core platform.

The scope explicitly excludes real-time video processing, model training from scratch (as pre-trained or quantised models are used throughout) and mobile or embedded deployment in this phase.

2. Literature Review and Problem Formulation

This chapter presents a critical review of the relevant literature on deepfake generation and detection, and formulates the research problem that the project seeks to address.

2.1 Background Theory

In this section, We will look more into various topics surrounding Deepfaked media and forensic methods and techniques.

2.1.1 Generative Adversarial Networks(GANs) and Deepfake Generation

In 2014, Ian J. Goodfellow introduced Generative Adversarial Network (GAN). It is a framework in which there is a generator network learns to create altered data by competing against a discriminator network which tries to differentiate between real and generated samples.

GANs rapidly became the dominant neural network architecture for high-fi synthetic face generation. Some key variants of the GAN are Progressive GAN and StyleGAN, which produce realistic human faces with high resolution. Also, face-swapping deepfake methods use encoder-decoder architectures. These separate the subject’s facial features from their pose and expression. This allows the overlaying of one person's face onto another's video.

As of late, diffusion-based generative models have demonstrated superior sample quality and training stability compared to GANs. The result is a landscape in which synthetic face generation is accessible, fast, and increasingly difficult to detect by visual inspection alone.

2.1.2 Vision Language Models for Forensics

Large multimodal models such as the Qwen3-VL series combine visual encoding with language modelling, enabling them to reason about image content through natural language chains of thought. When prompted with a structured forensic task, such models can identify texture anomalies, blending boundaries, and geometric inconsistencies in a

way that explicitly exposes their reasoning. VisionX exploits this capability by prompting Qwen3-VL-4B (to perform chain-of-thought analysis before outputting a structured fake probability score, providing both a decision and a human-readable rationale.

2.1.3 Audio Deepfake Detection

Voice synthesis and voice conversion have advanced substantially with neural vocoder architectures. Mel-Frequency Cepstral Coefficients (MFCCs) have been the long-standing standard feature representation for speech processing. MFCCs compress the spectral envelope of speech into a compact, perceptually-motivated feature vector, making them effective inputs for both speaker recognition and anti-spoofing classifiers. The ASVspoof challenge series has established standardised benchmarks for evaluating audio deepfake detection systems.

2.1.4 Temporal “Watermarking”

The rapidly improving quality of AI generated videos will eventually make current day AI based classification of videos obsolete, so a new deterministic method/protocol to filter them needs to be developed. The original methods of adding metadata to the video or adding a simple watermark are obsolete due to the ease of removing them with today’s technology. Temporal watermarking alone also fails because identification of the pixels allows them to be identified by averaging the frames. By encrypting the tag, it is possible to identify the presence of a tag but impossible to distinguish it from noise in the video.

2.2 Critical review of literature:

Ref.	Title and Year	Methodologies	Dataset if applicable	Limitations	Observations
[1]	**Generative Adversarial Nets, NeurIPS (2014)**	Foundational GAN framework: generator vs. discriminator adversarial training for realistic data synthesis.	MNIST, CIFAR-10, Faces	Training instability Requires careful learning rate balancing. No explicit temporal modelling.	Foundational theory for all GAN-based deepfakes. The entire rationale for forensic anomaly detection traces back to GAN artifacts described in this work.
[2]	**A Style-Based Generator Architecture for GANs (StyleGAN) CVPR, 2019**	Style-transfer-inspired GAN for high-resolution photorealistic face synthesis; AdaIN, progressive growing.	*FFHQ, CelebA - HQ*	Characteristic grid artefacts from transposed convolution upsampling visible in FFT spectrum. Does not model temporal sequences.	StyleGAN is a dominant architecture in face-swap deepfakes that FaceForensics++ benchmarks against and is essential for FFT Frequency Analysis
[3]	**High-Resolution Image Synthesis with Latent Diffusion Models, CVPR 2022**	Latent diffusion models operating in autoencoder latent space; text-to-image synthesis at scale.	*LAION-400M, COCO, LSUN*	Latent space diffusion leaves subtler spectral artefacts than GANs. Traditional GAN-trained detectors fail on diffusion outputs.	Establishes Stable Diffusion as the main modern deepfake threat. Diffusion-generated images require updated forensic methods motivating VLM-based reasoning approach.

[4]	**FaceForensics ++: Learning to detect manipulated facial images.** (2019)	Automated generation of facial manipulations using DeepFakes and CNN classification.	Face Forensics ++	Lacks an integrated audio signal. Performance greatly degrades on compressed videos.	Established a gold-standard benchmark for four distinct types of facial manipulation
[5]	**MediaPipe: framework for perceiving / processing reality (2019)**	Used for high-speed, 468-point face landmark tracking.	Proprietary Google Datasets	Performance fluctuates in extreme low-light environments or heavy occlusions.	Capable of tracking 468 landmarks at 47+ FPS, enabling real-time biological signal monitoring
[6]	**On the Detection of Synthetic Images Generated by Diffusion Models, ICASSP** (2023)	Forensic analysis of diffusion model artifacts. Extends GAN-trained detectors to latent diffusion images.	*LDM (200K), BigGAN, DALL·E 2, GLIDE; real: COCO, LSUN, ImageNet*	GAN-trained detectors suffer a 20–30% accuracy drop on diffusion images. Compression and resizing degrade spectral fingerprints. No audio integration.	It shows that diffusion models leave detectable spectral patterns despite evading GAN detectors. Justifies FFT analysis signal forensics.
[7]	**Exploring Temporal Coherence for More General Video Face Forgery Detection, ICCV** (2021)	Temporal coherence modeling for video deepfake detection; inter-frame consistency analysis.	*FaceForensics++, Celeb-DF, DFDC*	Needs training on video-format data; Doesn't handle single-image or audio channels.	Elaborates on sequential frame-by-frame VLM analysis in Video Mode

[8]	**Comprehensive Review of ML Techniques for Deepfake Detection** (2025)	Systematic comparison of CNN architectures (Xception, ResNet, VGG) under real-world conditions.	DFDC, FaceForensics++	Detection accuracy drops up to **50%** when deployed in real-world environments compared to lab tests.	Single mode systems (video-only or audio-only) are increasingly vulnerable to sophisticated, synchronized forgeries.
[9]	**AASIST: Audio anti-spoofing w/ integrated spectro-temporal graph attention networks.** (2021)	Integrated Spectro-Temporal Graph Attention Networks for audio anti-spoofing.	ASVspoof 2019 / 2021	Highly sensitive to channel noise and specific microphone hardware signatures	Superior at detecting "spectral fingerprints" and artifacts left by neural vocoders
[10]	**ASVspoof 2021: accel. progress in spoofed deepfake speech detection** (2021)	Benchmarked audio anti-spoofing robustness against speech compression and channel variability	ASVspoof 2021	Performance of audio detectors can degrade significantly on unseen or newer voice cloning systems	Highlighted the need for detecting spectral fingerprints and vocoder-level artifacts in synthetic speech
[11]	**Qwen3-VL Technical Report** (2025)	Architecture of Qwen3-VL; DeepStack multi-level feature fusion; 256K-token	DocVQA (95.3%), ScreenSpot (94.0%), MMBench (85.1%),	4B parameter variant requires ~4 GB VRAM under 4-bit GGUF quantisation. Chain-of-thought	The primary image/video classification model in VisionX. Many specific architectural

		context; thinking/reasoning variants.	AI2D (84.1%); zero-shot across 39 languages	reasoning adds inference latency. Not fine-tuned on deepfakes.	choices justified by this report.
[12]	**Large-Scale Contrastive Language-Audio Pretraining with Feature Fusion and Keyword-to-Caption Augmentation (LAION-CLAP),** (2023)	LAION-CLAP: contrastive audio-text joint embedding using HTS-AT encoder. Zero-shot audio classification.	*LAION-Audio-630K, AudioCaps; zero-shot: ESC-50 (82.6%), CLOTHO (59.3%)*	Zero-shot framing limits detection of highly specialised novel vocoders. No temporal localisation within clips. Fine-tuning on deepfake-specific data would boost performance.	The primary audio model in VisionX. The HTS-AT encoder, 512-D embedding space, 48 kHz requirement, The zero-shot accuracy on ESC-50 was a primary factor for zero-shot usage.
[13]	**How to Share a Secret, Shamir Adi** (1979)	(k,n)-threshold secret sharing using polynomial interpolation over finite fields	*N/A*	It does not inherently provide authentication. Vulnerable to malicious participants.	The Shamir Secret Sharing scheme is a core algorithmic component of VisionX's video watermarking pipeline
[14]	**ONNX-based runtime performance analysis: YOLO, Resnet** (2021)	Hardware-accelerated inference engine supporting model quantization.	N/A	Quantization to INT8/4 formats can lead to minor losses in forensic classification precision	Indispensable for achieving the sub-50ms latency needed for real-time forensic applications

[15]	**Evolving from Single-modal to Multi-modal Facial Deepfake Detection: Progress and Challenges** (2024)	Survey of deepfake detection from single-modal to multi-modal; role of VLMs and MLLMs; diffusion model challenges.	*FaceForensics++, Celeb-DF, DFDC, DGM4, DeepFake-Eval-2024*	Survey inherits limitations of reviewed papers; VLM-based detection requires empirical deepfake-specific validation at scale. No single dataset covers all manipulation types.	The most current (2024-2025) survey directly validating architectural choices. Confirms that the field is converging toward VLM-integrated multi-modal detection.

Table 2.1: Summary of Reviewed Literature

2.3 Problem Formulation:

Here we look at the various shortcomings that were found to be present with current systems after going through the existing literature.

2.3.1 Research Gaps

Based on the review, the following gaps are identified:

- **Privacy gap:** All production-grade deepfake detection tools reviewed operate as cloud services, requiring media upload to remote servers. This is incompatible with privacy-sensitive use cases.
- **Multi-modality gap:** Image and audio deepfake detection are treated as separate research problems. No unified, locally-deployed platform handles both.
- **Signal integration gap:** While individual forensic signals (EXIF, ELA, CFA, FFT) have been studied independently, their systematic integration into a single composite verdict pipeline is rarely addressed in open-source tools.

- **Explainability Gap:** Most detection systems return a verdict without interpretable reasoning. VLM-based classification can expose the model's rationale, improving trust and utility.
- **Digital Signature Integrity Gap:** Most systems use metadata-based tagging or use standard watermarks to label a video. Modern methods can easily get rid of these tags.

2.3.2 Research Questions

The following research questions guide this project:

- **RQ1:** Can a combination of different input types be handled by the same platform while achieving reliable deepfake detection when deployed entirely on the client device?
- **RQ2:** What is a good scoring strategy for combining different forensic signals into a single understandable verdict?
- **RQ3:** How can a video be tagged in such a way that it is impossible to remove the tag without destroying the original video?

3. Problem Statement

This chapter formally defines the aim and objectives of VisionX, the scope of the investigation, and the methods and methodology employed to attain each objective.

- **Title**

  Vision X: Multi-Modal Deepfake Detection — A Privacy-First Edge AI Platform

- **Aim**

  To design, develop, and validate a secure and privacy protecting, edge-deployed multi-modal deepfake detection platform. It combines a vision-language model classifier with multi-signal image and audio forensics, running entirely on the user's local machine without transmitting any media data to external servers.

- **Objectives**

  Here are the various objectives that we have set as our goals:

  - **Objective 1:** To implement an end-to-end image and video deepfake detection pipeline combining MediaPipe facial landmark extraction, Qwen3-VL-4B VLM classification (with chain-of-thought reasoning), and a four-signal forensic engine (EXIF, ELA, CFA, FFT) with composite verdict scoring.
  - **Objective 2:** To develop an audio deepfake detection pipeline utilizing the LAION-CLAP framework for high-resolution spectrogram processing and HTS-AT embedding generation, enabling robust segment-level analysis and overall confidence reporting.
  - **Objective 3:** To design and implement a FastAPI backend with Pydantic schema validation that serves all analysis pipelines and provides health monitoring for deployed models.
  - **Objective 4:** Design a system to embed a temporal encrypted “watermark” to tag AI generated images.

- **Objective 5:** To build a React 19 frontend providing an intuitive interface for image, audio, and video analysis modes, including forensic heatmap visualisation, detailed verdict displays and the Xite AI assistant for forensic Q&A.
- **Objective 6:** Evaluate the detection accuracy and system performance of the implemented pipelines, and define a quantitative evaluation framework for future extension.

- **Scope of Present Investigation**

The present investigation encompasses the full implementation of the image analysis pipeline (Objective 1), the fully functional audio pipeline backend and frontend (Objective 2), the FastAPI backend architecture (Objective 3), the React frontend for image and audio modes (Objective 4), and preliminary performance characterisation (Objective 5).

- **Methods and Methodology/Approach to attain each objective**

Objective No.	Statement of the Objective	Method/ Methodology	Resources Utilised
1	**Image deepfake detection pipeline**	Two-stage pipeline: MediaPipe landmark detection followed by Qwen3-VL-4B 4-bit GGUF VLM via llama-cpp-python; forensic signals aggregated via weighted composite scoring	MediaPipe, llama-cpp-python, Qwen3-VL GGUF, OpenCV, NumPy, custom FindingsEngine
2	**Audio deepfake detection pipeline**	High-resolution spectrogram processing and HTS-AT embedding generation via the CLAP framework; segment-level	librosa, NumPy, LAION-CLAP, custom AudioPreprocessor and AudioClassifier

		analysis and average-pooled overall confidence scoring.	
3	**RESTful FastAPI backend**	FastAPI REST API with Pydantic v2 schema validation; modular service injection at startup; health monitoring endpoint	FastAPI , Uvicorn , Pydantic , python-multipart
4	**React frontend**	Component-based UI with three analysis modes; native fetch() for API calls; recharts for data visualisation; single-file CSS variable theming	React 19, Vite, recharts, TailwindCSS
5	**Performance evaluation**	Benchmark suite measuring per-image execution time; pytest-cov for code coverage; API integration tests; manual evaluation on labelled test images	pytest, pytest-cov, benchmark.py

Table 3.1 Objective and the respective methodologies

4. Problem Solving

This chapter describes the architecture, design and implementation details of VisionX. The system comprises of a backend using FastAPI to create API enpoints, Llama and ONNXRuntime for machine learning model inferencing, and OpenCV and Librosa for pre-processing. A frontend using Vite for setup and React.js for DOM and webpage design. Uvicorn is used as a lightning-fast ASGI (Asynchronous Server Gateway Interface) web server to connect the frontend to the backend.

4.1 System Architecture

The high-level architecture of VisionX follows a client-server pattern constrained to the local machine which means during deployment everything runs on the user's hardware without the need for cloud services. The browser-based frontend communicates with the backend via HTTP FormData requests. The backend delegates analysis tasks to a modular services layer composed of independently instantiated service objects.

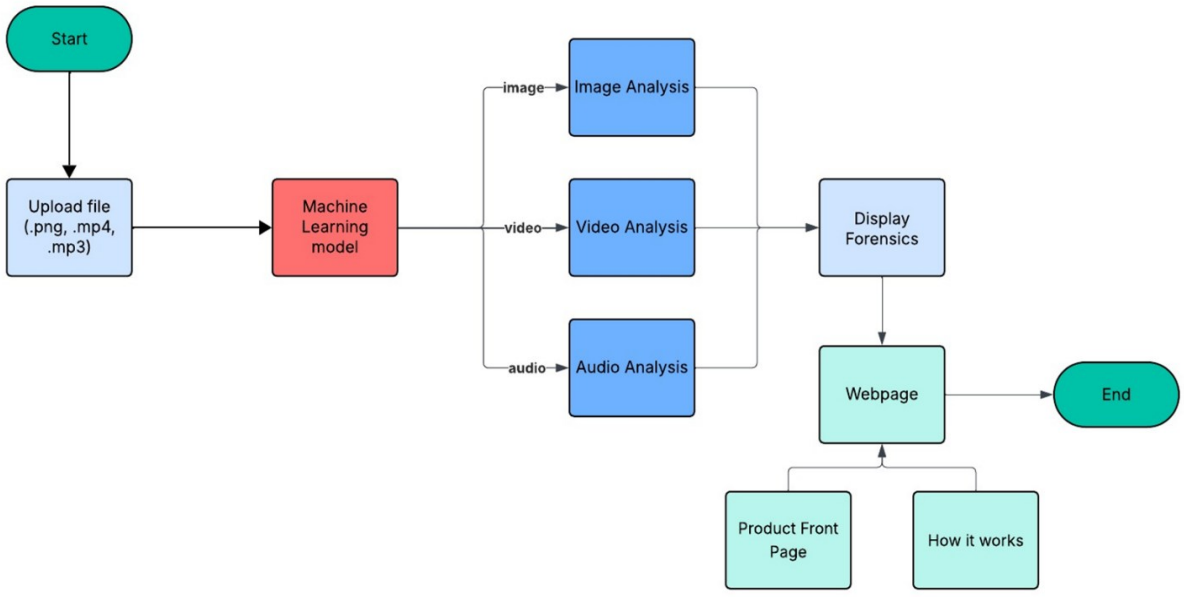


Figure 4.1 End-to-end system architecture

To manage the large memory footprint of the Qwen3-VL-4B model, we have implemented a lazy-loading strategy using lru_cache decorator, which will dynamically load which ever models are currently being used. Image, Video and Audio services are mutually exclusive in memory, followed by a garbage collection pass. This prevents out-of-memory conditions on hardware without intervention from the user.

4.2 Backend Design

Here we take a look in detail at how the backend is structured in our project:

4.2.1 Technology Stack

Package	Purpose
FastAPI	HTTP framework; asynchronous request handling
Uvicorn	ASGI server
Llama-cpp-python	Local inference for Qwen3-VL-4B model
ONNX Runtime	ML model inference on CPU
ONNX	ONNX model format support
MediaPipe	468-point facial landmark detection
OpenCV	Image decoding, resizing, and preprocessing
NumPy	Array operations throughout the pipeline
Librosa	Audio loading, resampling, MFCC feature extraction
Pydantic	Request/response schema validation and serialisation
python-multipart	Enables file upload parsing in FastAPI
python-dotenv	.env environment variable support
pytest+pytest-cov	Unit testing and coverage reporting

Table 4.1 Python Dependencies

4.2.2 API Routes

The backend exposes the following REST endpoints:

Endpoint	Method	Description
/analyze/image	POST	Accepts JPEG/PNG (≤10 MB). Runs face detection, VLM classification, and FindingsEngine. Returns per-face confidence, label, VLM reasoning, and landmark geometry.
/image/forensics	POST	Accepts any image plus optional face bounding box. Runs EXIF, ELA, CFA, FFT analyses. Returns per-signal scores and composite verdict.
/image/chat	POST	Accepts a user prompt, VLM forensic reasoning text, and conversation history. Returns a Xite AI assistant reply using the VLM.
/analyze/audio	POST	Accepts WAV/MP3. Runs MFCC preprocessing and segment-level CNN classification. Returns segment timeline and overall verdict.
/audio/forensics	POST	Accepts WAV/MP3. Returns spectral and prosody forensic signals. (Placeholder logic — full implementation pending.)
/health	GET	Returns system health status and loaded model identifiers.
/analyze/video	POST	Accepts .MP4 / .MOV. Extracts frames from video and submits to image analysis VLM. Returns confidence score with reasoning.

Endpoint	Method	Description
/watermark/detect	POST	Accepts .MP4 / .MKV. Analysis the video and detects whether it contains the invisible watermark or not.
/watermark/embed	POST	Accepts .MP4 / .MKV. Embeds the invisible watermark into the uploaded video.

Table 4.2 REST API Endpoints

4.3 Image Analysis Pipeline

The image analysis pipeline accepts a single JPEG or PNG upload and processes it through three sequential stages: face detection, VLM classification, and forensic analysis.

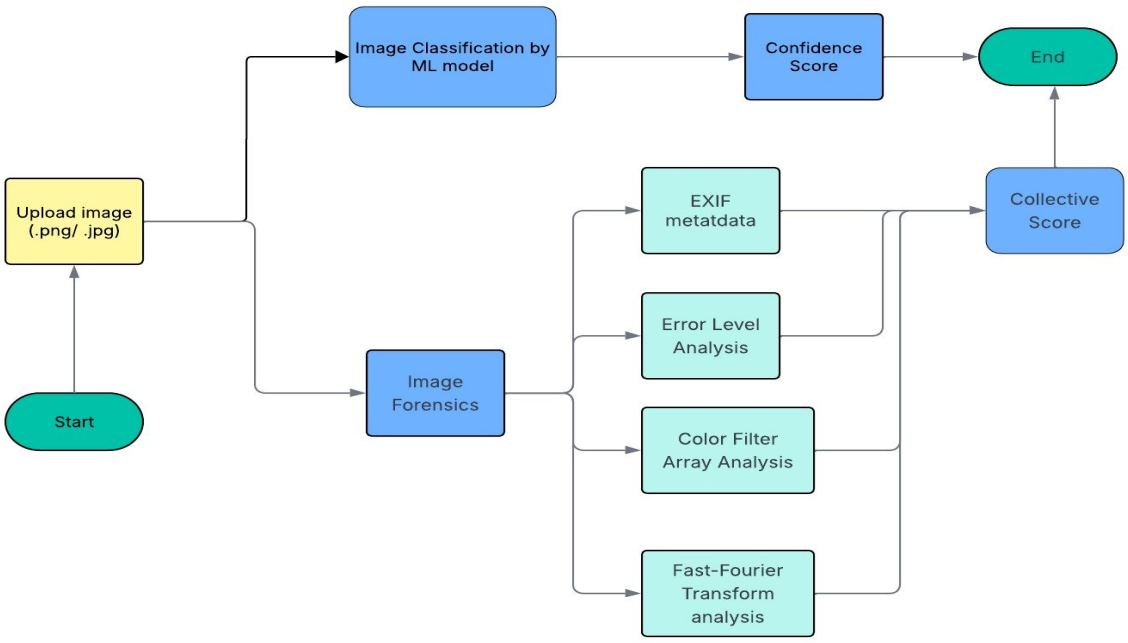


Figure 4.2 Image Analysis Pipeline — ML Classification and Four-Signal Forensics

4.3.1 Face Detection

Performed by the FaceDetector service, which wraps MediaPipe's Face Landmarker task file. For each detected face, the service returns 468 facial landmark coordinates in pixel space, a padded bounding box (20% padding on each side), and a detection confidence score. The landmark model additionally enables extraction of geometric features including inter-eye distance, eye asymmetry percentage, mouth width, and mouth aspect ratio — all reported as raw measurements in the API response for use by the frontend. This service can detect up to four individual faces which are passed to the Image model for per-face classification.

4.3.2 VLM Classification (Qwen3-VL-4B)

Each detected face crop is extracted from the original image using the padded bounding box, padded to a square aspect rati o to avoid distortion, and resized to 224 × 224 pixels. The crop is then Base64-encoded and submitted to the Qwen3-VL-4B model via the ImageClassifier service.

The model is loaded using llama-cpp-python with the Llama and Qwen3VLChatHandler native classes, enabling vision inputs via the multimodal projector. Inference runs with a context length of 8,192 tokens, flash attention enabled, and forced reasoning mode active. The system prompt instructs the model to act as a deepfake detection agent, it looks for various ways to detect what type of image is provided and conclude with a markdown JSON block containing a single key, fake_probability, with a float value in [0.0, 1.0].

The response is parsed to extract the chain-of-thought reasoning (from the <think> tag) and the JSON confidence score. This reasoning text is stored in the FaceResult response as the reasoning field, where it serves as context for the Xite chat assistant. The chatbot is intentionally loaded without the multi-modal projector file, which means that the chatbot cannot ‘see’ the image. This is done so that chatting feels fast and responsive, a

pivotal decision that makes the chatbot much are lightweight. Robust fallback logic handles cases where the model response is malformed or truncated. Classification thresholds are: score < 0.30 → real; 0.30–0.60 → uncertain; score > 0.60 → fake.

4.3.3 Forensics Pipeline

The forensics pipeline is invoked via the POST /image/forensics endpoint and runs four independent signal analyses, each producing a severity rating (anomalous, suspicious, or normal) and a numerical score [0–100]:

- **EXIF Metadata Analysis:** Examines the Exchangeable Image File Format header of the uploaded image. AI-generated images typically lack camera metadata or contain inconsistent field values. Missing EXIF data is scored highly anomalous; partial metadata yields a suspicious rating.
- **Error Level Analysis (ELA):** Re-compresses the image at a fixed quality level and amplifies the per-pixel difference between the original and re-compressed versions. AI-generated face regions exhibit spatially uniform or atypically low error levels compared to authentic photographs. The ELA heatmap is returned as a base64-encoded JPEG for display in the frontend.
- **Camera Fingerprint Analysis (CFA):** Analyses the spatial noise pattern of the image. Real camera photographs exhibit a consistent sensor noise field; GAN upsampling layers disrupt this pattern, creating detectable statistical anomalies. The analysis may be applied to the full image or restricted to the face bounding box region.
- **FFT Frequency Analysis:** Computes the Fast Fourier Transform of the image and examines the magnitude spectrum for periodic grid artefacts introduced by GAN transposed convolution layers. The presence of anomalous spectral peaks above a threshold indicates synthetic generation.

The composite verdict is computed as follows: if the aggregate weighted score is ≥ 68, or two or more signals are rated anomalous, the verdict is LIKELY AI / DEEPFAKE. If the score is ≥ 38 or two or more signals are rated suspicious, the verdict is SUSPICIOUS. Otherwise, the verdict is LIKELY AUTHENTIC. When no face is detected, only EXIF analysis is performed and the fake threshold tightens to ≥ 70. These thresholds are summarised in Table 4.4.

Verdict	Condition
LIKELY AI / DEEPFAKE	Composite score ≥ 68 OR two or more signals rated anomalous
SUSPICIOUS	Composite score ≥ 38 OR two or more signals rated suspicious
LIKELY AUTHENTIC	Neither of the above conditions met
NO FACE DETECTED	When MediaPipe fails to detect faces; Only EXIF runs.

Table 4.3 Forensics Verdict Thresholds

4.3.4 FindingsEngine and Xite Chat

The FindingsEngine generates plain-English explanatory strings for each analysed face, combining the ML confidence score with the landmark geometry measurements. These strings are intended to make the system's reasoning transparent to non-expert users. Structural data — including inter-eye distance in pixels, eye asymmetry percentage, mouth width, and landmark completeness — is also returned as raw numerical values, allowing the frontend to present visual summaries without additional processing.

The Xite chat assistant is a draggable floating panel in image mode. It sends user questions to the POST /image/chat endpoint, which constructs a prompt from the user's message, the VLM's forensic reasoning chain and a rolling history of the last six conversation turns. The VLM responds in a conversational tone, drawing on the forensic context only when relevant to the user's question. This feature allows non-expert users to interrogate the detection rationale interactively.

4.4 Audio Analysis Pipeline

The audio analysis pipeline accepts a WAV or MP3 upload and processes it through preprocessing, segmentation, feature extraction, and classification stages.

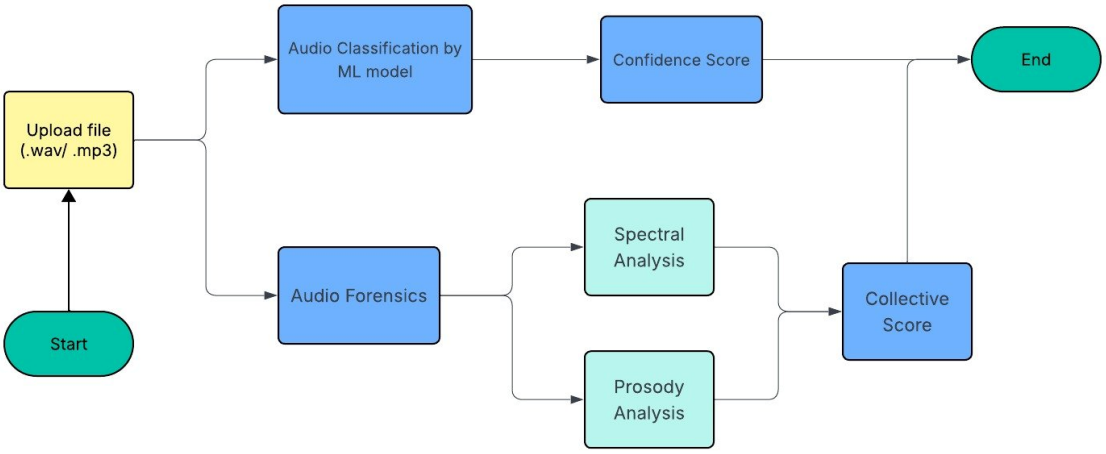


Figure 4.3: Audio Analysis Pipeline — Classification and Forensic Signal Analysis

4.4.1 Preprocessing

The AudioPreprocessor service handles the full audio loading pipeline in four stages step by step. Each step is implemented as a separate method/function to allow for independent testing and future replacement/maintenance.

- **Loading and Normalisation:** The uploaded audio file is stored in a temporary location and loaded via librosa in mono, ie the left and right channels of thr audio are merged into one middle channel. The waveform is then normalised to unit amplitude by dividing by the absolute peak value, ensuring a consistent range

regardless of source recording amplitude. Silent clips (all-zero waveforms) are unchanged when returned without triggering a division-by-zero error.

- **Resampling:** The normalised waveform is resampled to 48 kHz using librosa.resample method. This target rate is not arbitrary; the LAION-CLAP audio model operates optimally at 48 kHz and its core ClapProcessor computes mel-spectrogram features calibrated to this sample rate. Resampling to any other rate would result in the frequency bins of the mel filterbank not being aligned with the expected spectral coverage of the model. This results in a degraded inference quality. If the source audio is already at 48 kHz, the resample step is skipped.
- **Segmentation:** The full-length resampled waveform is sliced into fixed-length three-second chunks with 50% overlap (hop size = 1.5 s). Since the CLAP processor expects a uniform input length to construct the mel spectrogram, fixed-length segments are required. The 50% overlap ensures that transient artefacts at segment boundaries are captured by at least one segment. This reduces missed detections on short manipulated regions. The final segment is zero-padded to the standard length, if the remaining audio is shorter than three seconds.

The CLAP model's ClapProcessor internally computes a log-mel spectrogram from the raw waveform, a representation that preserves far richer spectral-temporal structure than a compressed 13-coefficient MFCC vector. The preprocessor therefore outputs raw NumPy waveform arrays, delegating all feature computation to the ClapProcessor at inference time.

4.4.2 Classification

The AudioClassifier service implements a zero-shot contrastive classification strategy using the LAION-CLAP framework, which is architecturally distinct from a conventional supervised CNN classifier. The classification pipeline operates as follows.

- **Model Loading:** At initialisation, the service loads three artefacts. First, the ClapProcessor from the local CLAP-htsat-fused directory (a HuggingFace-format

processor) is instantiated; this handles mel-spectrogram computation including the is_longer flag required by the model's feature-fusion module for long-form audio. Second, the ONNX export of the CLAP HTS-AT audio encoder (clap_audio_encoder.onnx) is loaded into ONNX Runtime with CUDA and CPU execution providers, enabling GPU acceleration when available. Third, a pre-computed NumPy archive (clap_text_embeddings.npy) containing the 512-dimensional text embeddings for the two class prompts ("real speech" and "fake or synthetic speech") is loaded, along with the learned logit scale scalar.

- **Inference:** For a given audio segment (or batch of segments), the ClapProcessor converts the raw waveform(s) into a padded input_features tensor (mel spectrogram). The is_longer flag is conditionally included if the ONNX session graph exposes that input node. ONNX Runtime runs the HTS-AT encoder forward pass, producing a 512-dimensional audio_features embedding per segment. The contrastive similarity between each audio embedding and the two text class embeddings is then computed as: logits = logit_scale × audio_features · text_features^{T}. A softmax is applied over the two logit values to yield class probabilities, and the probability assigned to the "fake" class index is returned as the segment-level confidence score in [0, 1].

- **Aggregation and Verdict:** A global inference pass on the full-length waveform yields the overall clip-level confidence score. A separate batched inference pass over all segments yields the per-segment timeline. Verdict thresholds applied to the overall score are: score > 0.60 → fake; score > 0.30 → uncertain; score ≤ 0.30 → real. The full per-segment timeline is returned in the API response, enabling fine-grained visualisation of temporal anomalies in the frontend strata chart. The batched classify() method accepts either a single NumPy array or a list of arrays, returning a scalar or a list of floats respectively, making the interface uniform for both global and segment-level calls.

4.5 Video Mode Pipeline

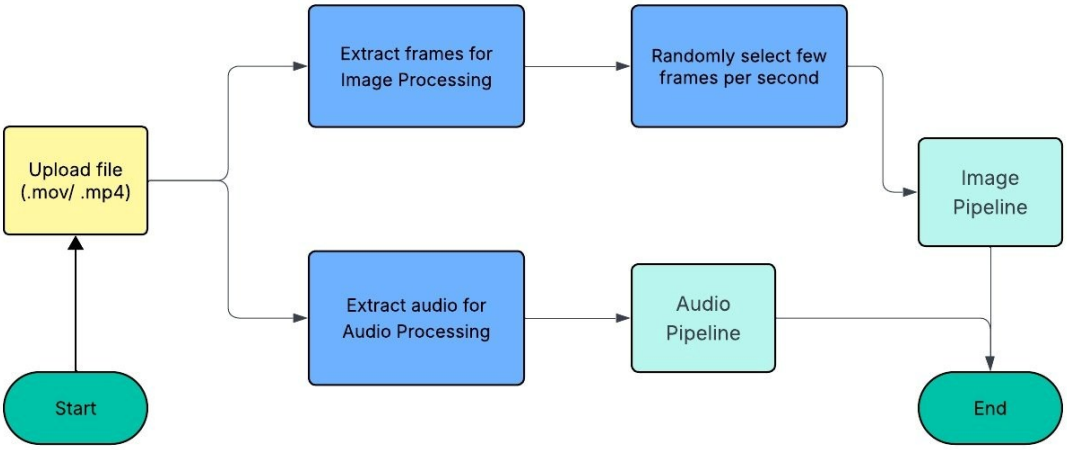


Figure 4.4 Video Analysis Pipeline — Frame Extraction and Parallel Audio/Image Processing

The video analysis pipeline accepts .mp4 or .mov uploads and produces a per-frame deepfake verdict by applying the shared Qwen3-VL-4B engine to facial crops extracted from the video stream. The pipeline is composed of three service classes, VideoPreprocessor, VideoClassifier, and VideoFindingsEngine, all instantiated via lru_cache and subject to the same mutual-exclusion memory management as the image pipeline.

4.5.1 Frame Extraction (VideoPreprocessor)

The VideoPreprocessor.extract_face_frames() method is the entry point for video analysis. The uploaded bytes are written to a temporary file. The file is opened via cv2.VideoCapture. The original frame rate is read from the container metadata; if it is zero or NaN, a fallback rate of 30 FPS is assumed. The total duration is computed as total_frames / orig_fps, and processing is capped at the video max duration of 10 seconds, limiting inference time on consumer hardware.

Frame sampling is governed by VIDEO_FPS = 2: a stride of round(orig_fps / 2) is computed so that exactly two frames per second are evaluated regardless of the source frame rate. For each sampled frame, the shared FaceDetector service is invoked. If at least one face is detected with a bounding box producing a crop of at least 50 × 50 pixels, the primary face crop is resized to 224 × 224 and appended to the extraction list along with its timestamp and frame index. Frames with no detected face, or with an undersized crop (e.g. subjects far from the camera), are discarded. The temporary file is deleted in a finally block after VideoCapture is released. If the extraction list is empty, the endpoint returns a face_detected: false response immediately without invoking the classifier.

4.5.2 Per-Frame VLM Classification (VideoClassifier)

The VideoClassifier wraps the same Llama/Qwen3-VL engine used by the image pipeline (shared via get_shared_vlm_engine()), avoiding a second model load. For each extracted face frame, the 224 × 224 BGR crop is JPEG-encoded via cv2.imencode and Base64-encoded into a data-URI string. The classification prompt instructs the model to output a strict JSON object with two keys: confidence (float [0.0, 1.0], where 1.0 = definitely fake) and reasoning (a brief natural-language explanation). The model response is parsed with a regex search for the first JSON object in the reply; if parsing fails, confidence defaults to 0.0 and an error flag is logged.

After all frames are processed, the average confidence across the frame set is computed. The peak confidence (highest single-frame score) is also tracked and included in the aggregate_reasoning string returned to the frontend. This per-frame sequential inference approach was chosen over batched processing because the Qwen3-VL model's chain-of-thought reasoning works better one image at a time, and because memory constraints on consumer hardware preclude loading multiple image crops simultaneously into the model context.

4.5.3 Verdict Generation (VideoFindingsEngine)

The VideoFindingsEngine.generate_findings() method maps the average confidence score to a human-readable verdict label and a structured list of detection events using the same REAL_THRESHOLD / UNCERTAIN_THRESHOLD configuration constants as the image pipeline (0.30 and 0.60 respectively). A score above UNCERTAIN_THRESHOLD produces a verdict of LIKELY AI / DEEPFAKE with three HIGH/MED severity events describing the nature of the VLM's findings. A score between the two thresholds returns SUSPICIOUS — REVIEW CAREFULLY with two MED severity events. A score below REAL_THRESHOLD returns LIKELY AUTHENTIC with two LOW severity events confirming visual consistency. The /analyze/video endpoint assembles the full response object including face_detected, frames_analyzed, aggregate_confidence, label, aggregate_events, aggregate_reasoning, frame_details (the per-frame timeline), execution_time_ms, and model_details identifying both the face model and the VLM variant.

The frontend VideoMode.jsx component maps the frame_details array to an AreaChart timeline (Recharts), displaying AI probability as a continuous waveform over time with a dashed 60% threshold line. The top five highest-confidence frames are surfaced as clickable "Key Evidence Frame" cards. Selecting a card opens a detail modal that displays the timestamp, per-frame score, severity tag, and the specific VLM reasoning string for that frame — giving the investigator a natural-language explanation for why that moment was flagged.

4.6 Video Watermarking

The inclusion of Video Watermarking in our project offers an alternative by which we can consistently identify AI generated content through a digital watermark that cannot be tampered with easily by attackers.

4.6.1 Encryption and Embedding

The embedder accepts any video and the tags to be embedded in the video. The tags are processed in a json file which is then converted to UTF-8 binary. This is then divided using "Shamir Secret Sharing" which enables the system to be resilient to loss of frames. The shares are then encrypted using AES encryption. These encrypted shares are then embedded in separate frames of the video so the message cannot be identified from any one frame.

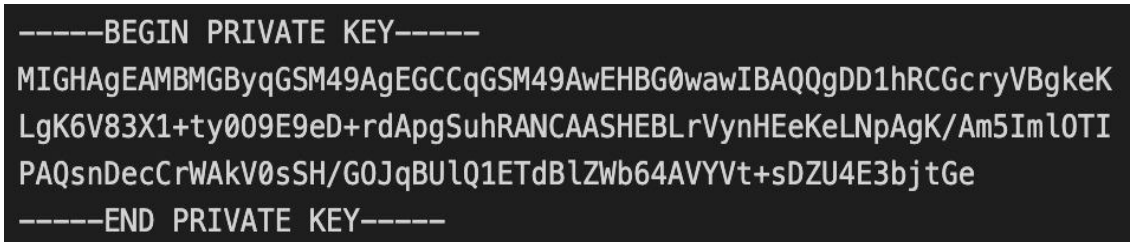


Figure 4.5 Private key generated for AES

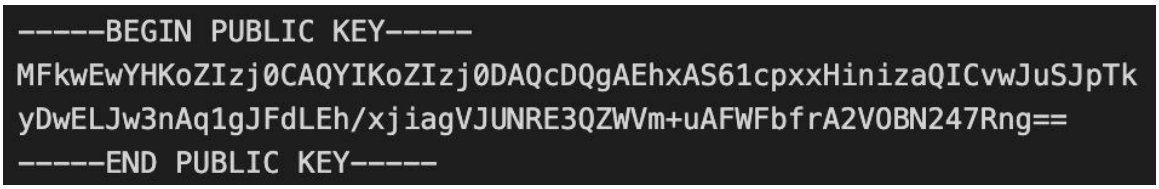


Figure 4.6 Public key generated for AES

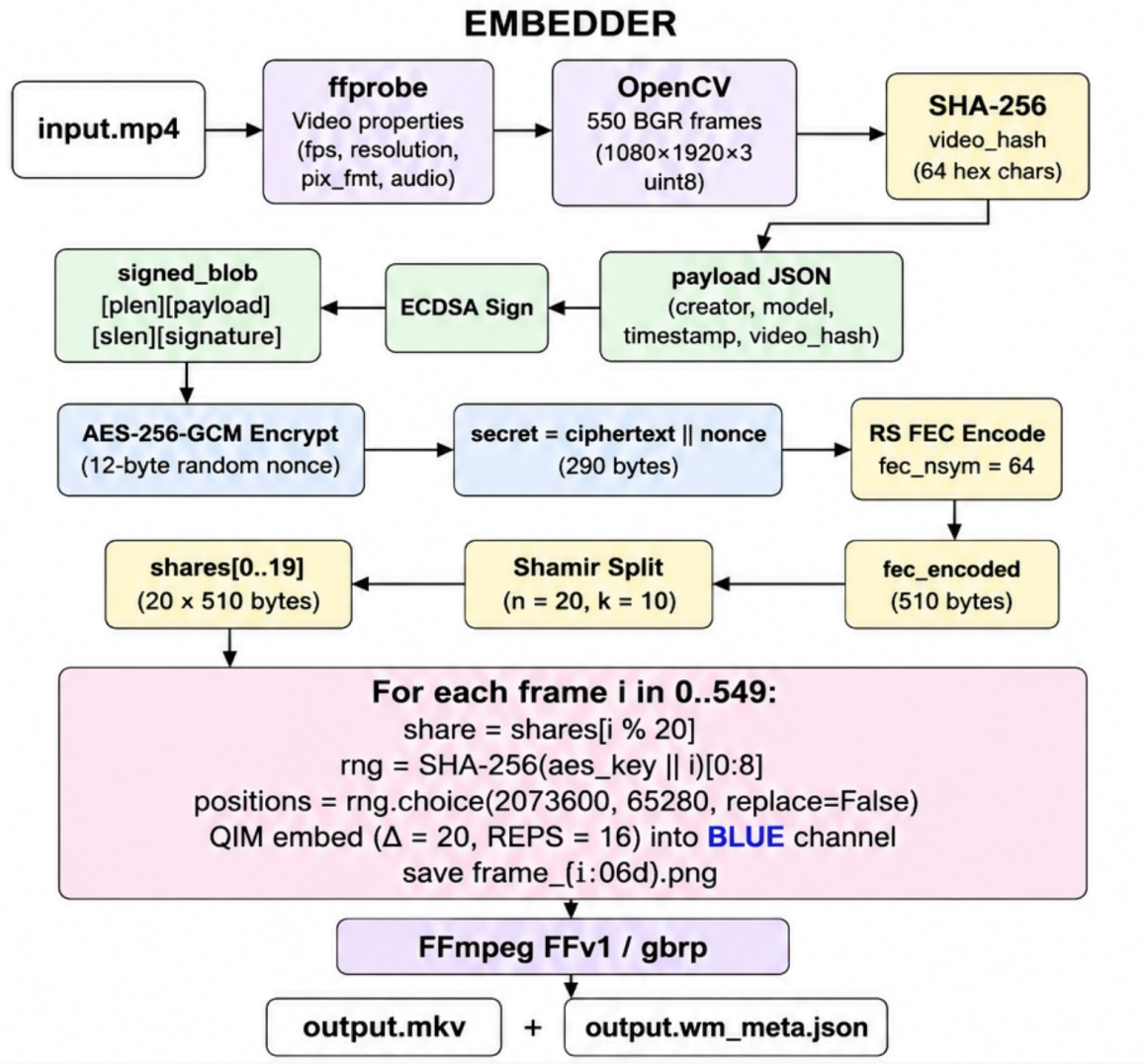


Fig 4.7 Embedder Flowchart

4.6.2 Detection and Decryption

The Detector knows which pixels to look at because the pixel location is calculated as a function of the frame count and the public key (which is used by the embedder to encrypt the message, ie same key is used for 2 purposes). It then looks for the minimum number of unique shares determined by Shamir Secret Sharing and decrypts them with the private key. The reconstructed message is then used to verify the video origin.

```
{
  "n_shares": 5,
  "n_frames_total": 550,
  "threshold": 2,
  "share_len": 510,
  "video_hash": "b692182bd8550898e8cd62cf089c445d0743c2282f665b081a0d6c3c3b55d596"
}
```


Figure 4.8 Specifications for Shamir Secret Sharing

```
=== Identification Result ===
✓ Watermark identified and verified successfully!
  Creator ID:  monkeyfoo
  Model ID:    gpt-4-vision
  Timestamp:   8.0s, 16.2s
  Video Hash: b692182bd8550898e8cd62cf089c445d0743c2282f665b081a0d6c3c3b55d596
  Frames used: 200/550, 400/550
```

Figure 4.9 Detection Result

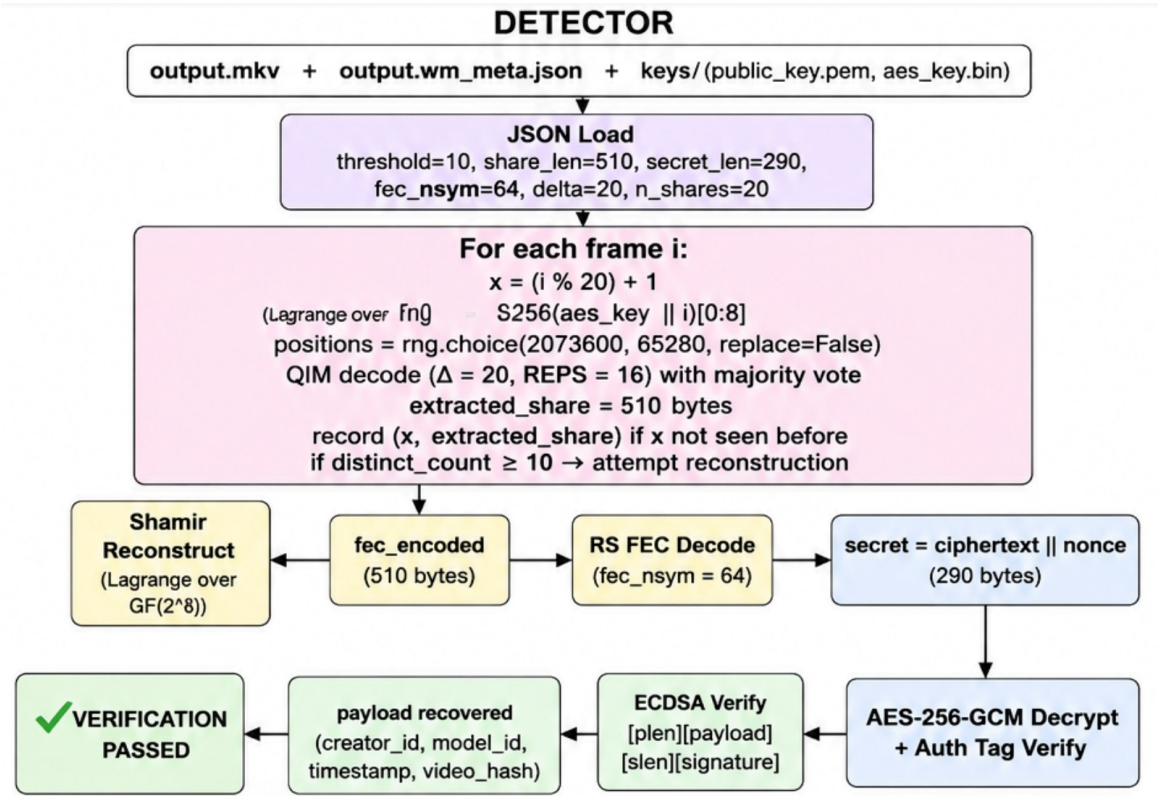


Fig 4.10 Detector Flowchart

4.7 Frontend Design

Here we take a look at how the Frontend works in our project:

4.7.1 Tech Stack

Package	Purpose
React	UI component framework
Vite	Build tool and development server
Tailwind CSS	Utility-first CSS framework
ESLint	Code quality and linting
Recharts	Composable React charting library
React-Markdown	Markdown rendering support
React Router DOM	Client-side routing
Axios	HTTP client for API requests

Table 4.4: Node.js Dependencies

4.7.2 Application Architecture

Here we see the directory structure and architecture of the project:

4.7.2.1 Directory Structure

```
src/
├── App.jsx                 # Main application component with routing logic
├── main.jsx               # React application entry point
├── index.css              # Global styling
├── common/                # Shared reusable components
│   ├── ConfidenceGauge.jsx
│   └── FileUploader.jsx
├── config/                # Configuration files
└── features/              # Feature-specific modules
    ├── image/             # Image analysis mode
    ├── audio/             # Audio analysis mode
    ├── video/             # Video analysis mode
    └── watermark/             # Watermarking features
```

Figure 4.11 Folder Structure

4.7.2.2 High-Level Architecture Diagram

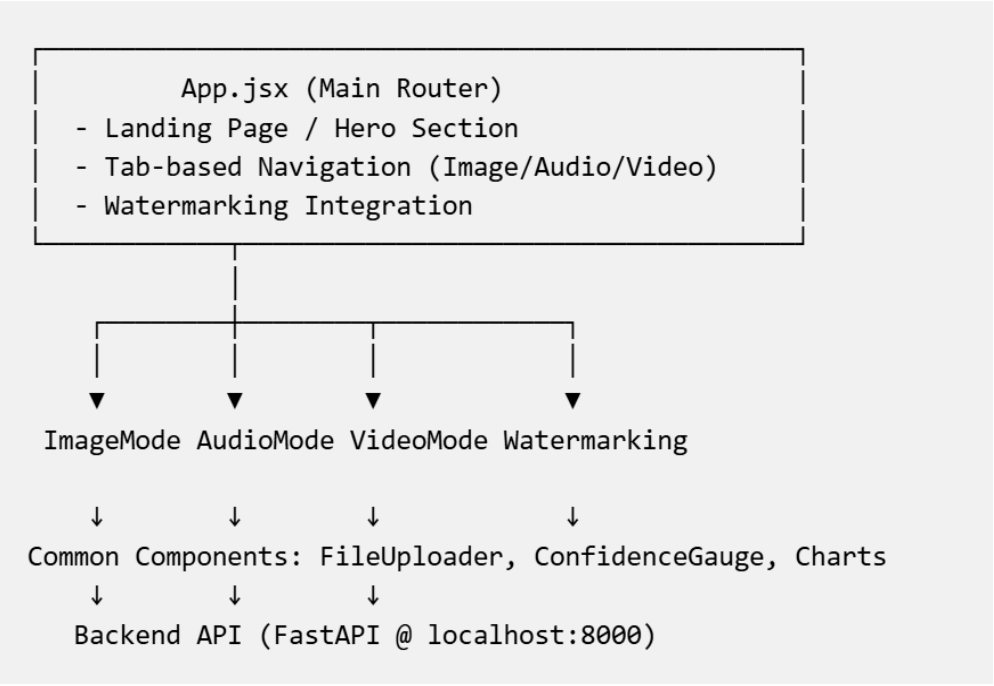


Figure 4.12 Frontend Flowchart

4.7.3 Core Components

Here we look in depth at the core components of the Frontend

4.7.3.1 App.jsx – Main Application Component

Purpose: This is the root application component that manages the overall state and navigation of the system.

Key features:

Landing Page: Hero section with the VisionX branding.

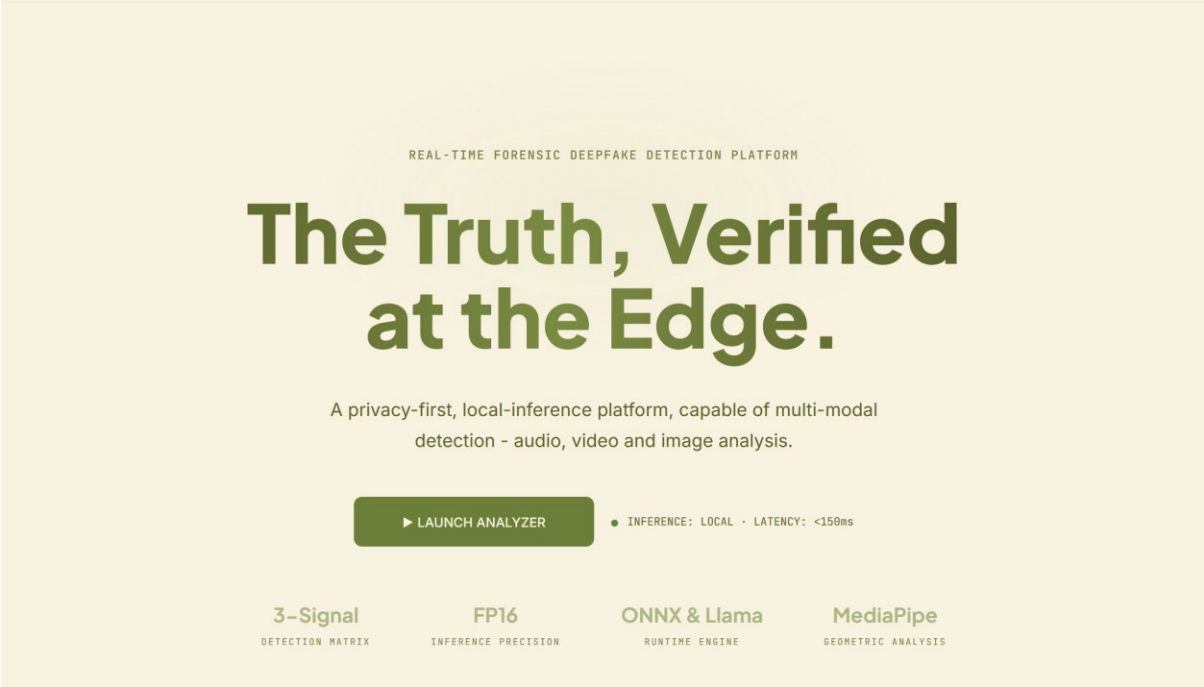


Figure 4.13 Landing Page

Tab-Based Navigation: Allows users to switch between Image, Audio and Video modes.

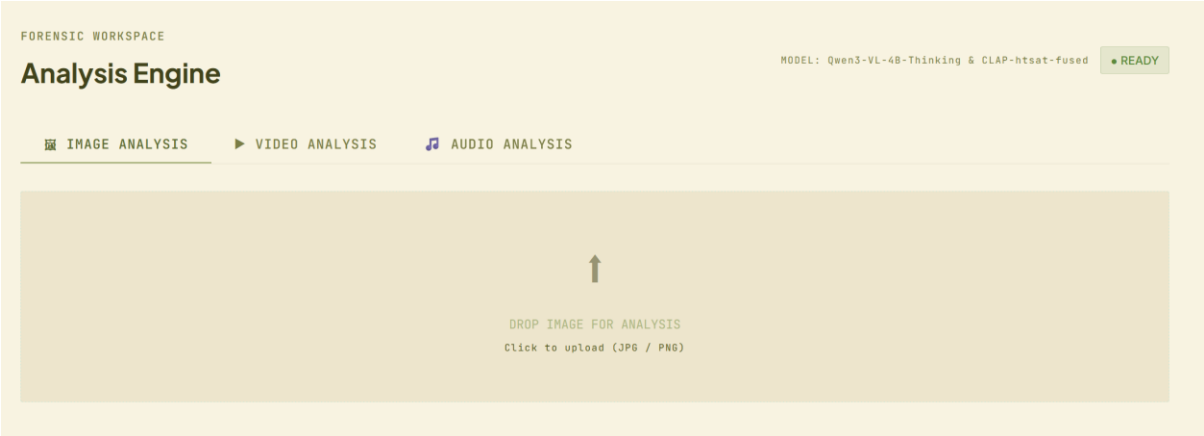


Figure 4.14 Media Modes

Fixed Navigation Bar: Animated logo, version display and 'SYSTEM ONLINE' status bar.

View Management: Handles transitions between main platform and watermarking section.

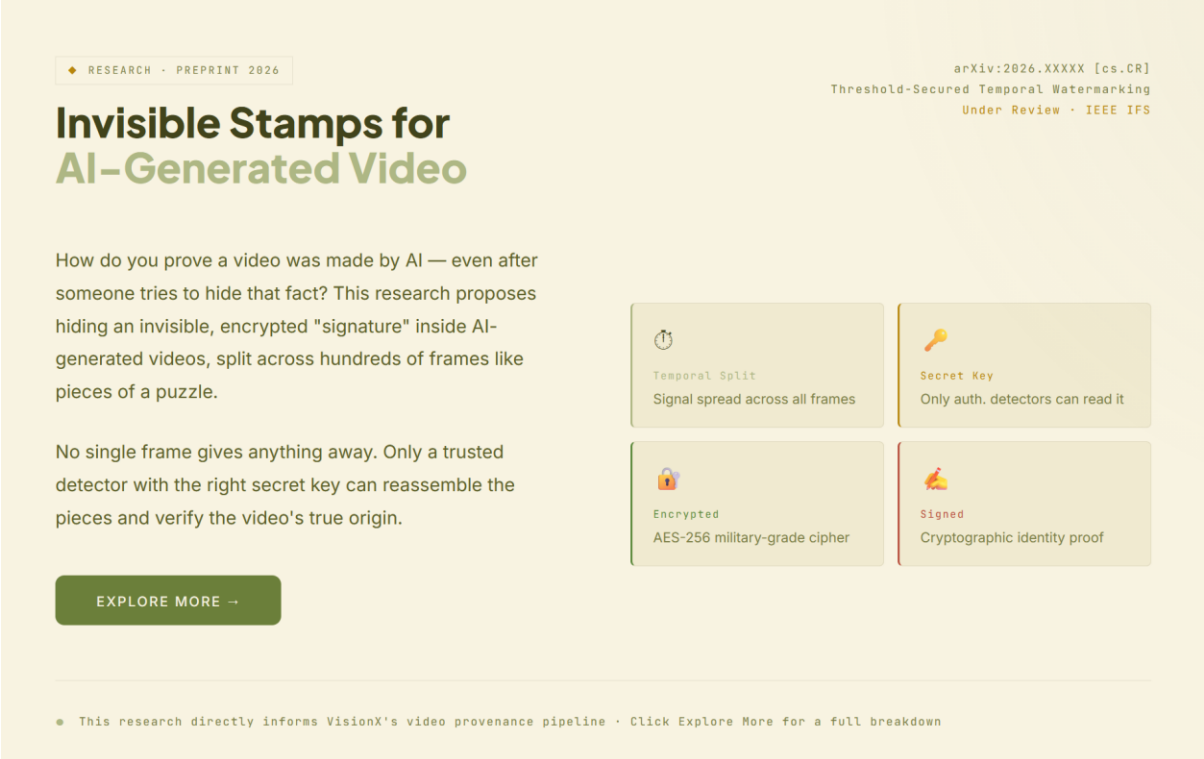


Figure 4.15 Watermark Intro

Custom Styling: Implements design tokens for consistent branding (colors, fonts, animations).

4.7.3.2 Image Mode

Purpose: Analyse images for deepfake detection with facial analysis capabilities.

Workflow:

1. User uploads image (PNG/JPEG)
2. System detects faces in image
3. Performs classification on each detected face
4. Generates forensic reasoning and confidence scores
5. Renders multi-face results with detailed analysis

Key Features:

Feature	Description
Face Detection	Identifies all faces in uploaded image
Multi-Face Support	Handles image with multiple faces
Confidence Gauge	Visual representation of deepfake probability
Forensic Analysis	Displays reasoning behind classification
Facial Mesh Mapping	MediaPipe overlay visualization
Heatmap View	Shows suspicious regions in image
Facial Landmarks	Visual analysis of facial features

Table 4.5: Image Mode

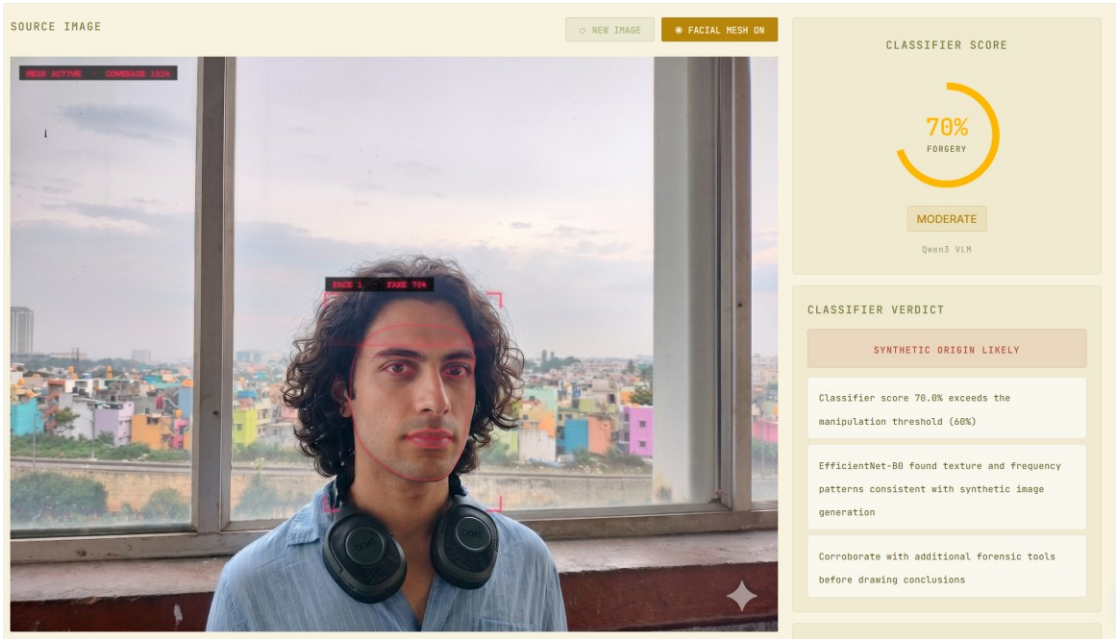


Figure 4.16 Image Mode output

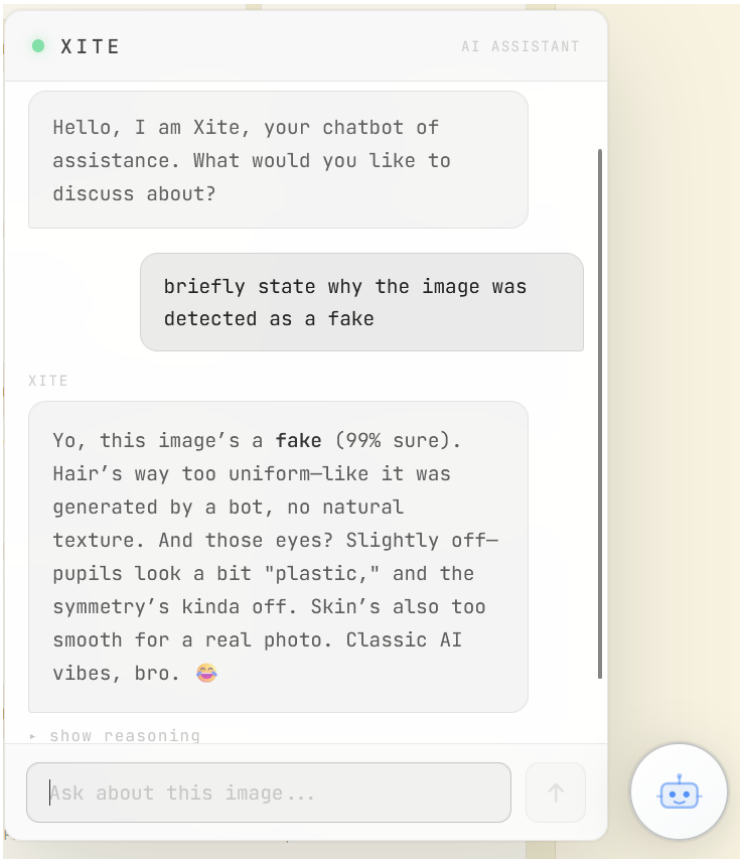


Figure 4.17 Image Mode Xite chatbot

4.7.3.3 Audio Mode

Purpose: Analyse audio files for voice deepfake detection

Workflow:

1. User uploads audio file (WAV, MP3, etc.)
2. System performs basic audio analysis
3. Optionally runs deeper forensic analysis
4. Displays results in dashboard

Key Features:

Feature	Description
Basic Analysis	Quick audio classification
Forensic Analysis	Detailed audio forensics (optional)
Results Dashboard	Comprehensive findings visualization
Processing Status	Real-time status updates
Error Handling	User-friendly error messages

Table 4.6: Audio Mode

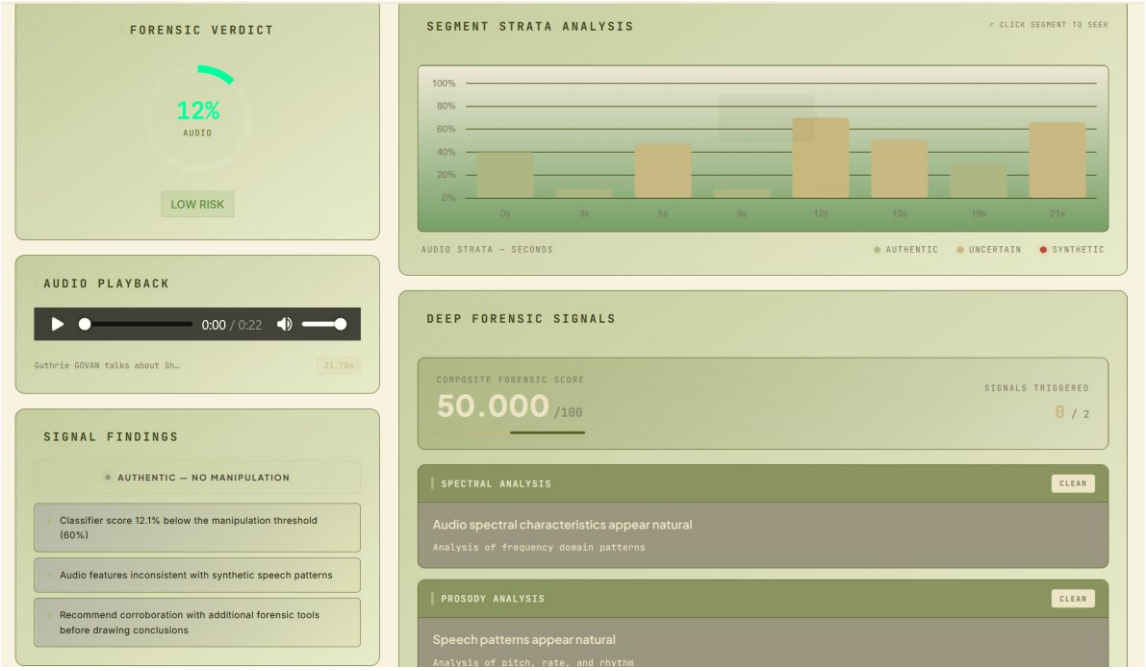


Figure 4.18 Audio Mode output

4.7.3.4 Video Mode

Purpose: Analyse videos frame-by-frame for deepfake detection.

Workflow:

1. User uploads video file
2. System extracts and analyses frames
3. Generates per-frame confidence scores
4. Produces timeline visualization
5. Identifies suspicious frames (keyframes)

Key Features:

Feature	Description
Per Frame Analysis	Processes each video frame
Timeline Visualization	Confidence scores over time

Keyframe Detection	Identifies most suspicious frames
Progress Tracking	Visual progress indicator
Statistics	Frame count and suspicious frame count
Event Logging	Detected anomalies and events

Table 4.7: Video Mode

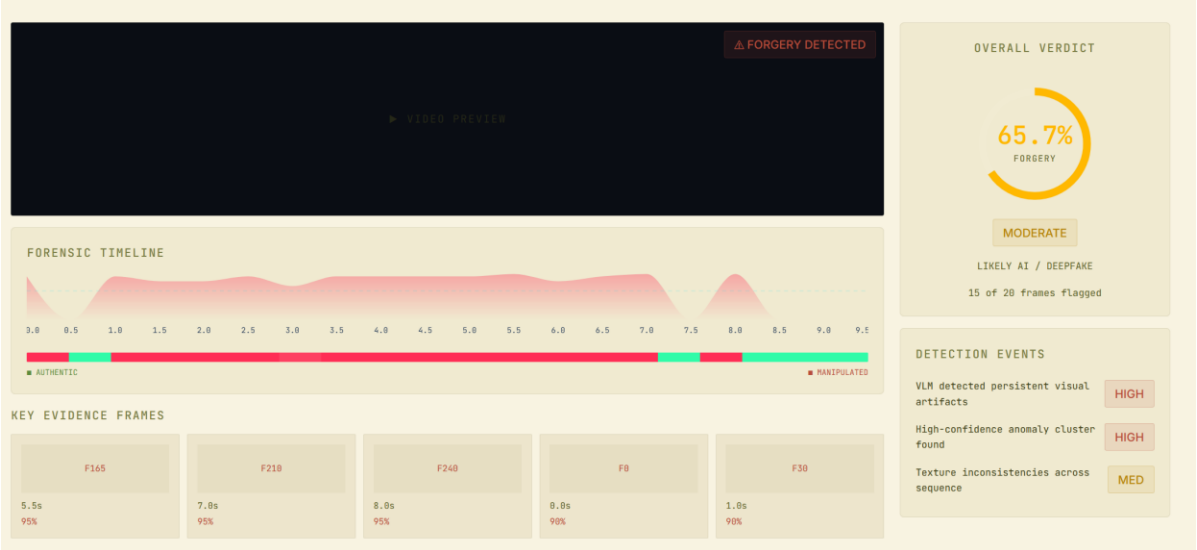


Figure 4.19 Video Mode output

Figure 4.20 Video Mode keyframe

4.7.3.5 Common Components

Here are the reusable components that are getting reused across different modes.

4.7.3.5.1 FileUploader.jsx

Purpose: Reusable file upload component with drag-and-drop support.

Features:

- Drag-and-drop functionality
- Click-to-upload intereface
- File type validation (configurable)
- Customizable title, subtitle, icon and height

4.7.3.5.2 ConfidenceGauge.jsx

Purpose: Visual representation of deepfake confidence score

Features:

- Circular gauge display
- Animated confidence needle
- Color-coded zones (Real/Uncertain/Fake)
- Percentage and label display

4.7.3.6 Watermarking Module

Purpose: Provides digital watermarking features for content authentication.

Components:

- WatermarkingIntro.jsx – Introduction to watermarking
- WatermarkingPage.jsx – Full watermarking interface

Features:

- Content watermarking workflow
- Authentication verification
- Integration with main platform

4.7.4 API Integration

Presented below are the API endpoint responses that the frontend receives from the backend

4.7.4.1 Image Analysis

```
POST /analyze/image
Content-Type: multipart/form-data

Request:
- image: File (PNG/JPEG)

Response:
{
  faces: [
    {
      confidence: number (0-1),
      label: "REAL" | "FAKE",
      landmarks: [...],
      heatmap: base64_image,
      reasoning: string
    },
    ...
  ],
  face_detected: boolean,
  error?: string
}
```

Figure 4.21 Image Mode API

4.7.4.2 Audio Analysis

```
POST /analyze/audio
Content-Type: multipart/form-data

Request:
- file: File (Audio)

Response:
{
  classification: "REAL" | "FAKE",
  confidence: number (0-1),
  mfcc_features: [...],
  spectrogram: base64_image,
  ...
}

POST /audio/forensics
Content-Type: multipart/form-data

Request:
- file: File (Audio)

Response:
{
  forensic_score: number,
  artifacts_detected: [...],
  compression_analysis: {...},
  voice_pattern: {...},
  ...
}
```

Figure 4.22 Audio Model API

4.7.4.3 Video Analysis

```
POST /analyze/video
Content-Type: multipart/form-data

Request:
- video: File (MP4/AVI/etc.)

Response:
{
  frame_details: [
    {
      frame_idx: number,
      timestamp: float,
      confidence: number (0-1),
      reasoning: string
    },
    ...
  ],
  label: "REAL" | "FAKE",
  aggregate_confidence: number (0-1),
  aggregate_events: [...],
  total_frames: number
}
```

Figure 4.23 Video Mode API

4.7.4.4 Error Handling

All endpoints return HTTP status codes with error details

- 200 – Success
- 400 – Bad request (invalid format)
- 422 – Validation error
- 500 – Server error

5. Results and Discussions

This chapter presents the quantitative results for the Qwen3-VL-4B-Thinking model, the CLAP-htsat-fused model and the novel watermarking system.

5.1 Image and Video Classification Performance

VisionX is powered by the 'Qwen3-VL-4B-Thinking' image classification model for the Image and Video mode. It is a part of the Qwen3 family which is undeniably one of the best open-source machine learning models that even competes with the industry standard models. This section first introduces the model within its published benchmark landscape, then presents our own evaluation results on a balanced real-world test set for our use-case.

5.1.1 Qwen3-VL: Architecture and Industry Benchmarks

VisionX is powered by the 'Qwen3-VL-4B-Thinking' image classification model for the Image and Video mode. It is a part of the Qwen3 family which is undeniably one of the best open-source machine learning models that even competes with the industry standard models. This section first introduces the model within its published benchmark landscape, then presents our own evaluation results on a balanced real-world test set for our use-case.

We use the **4.4B-parameter dense variant (Qwen3-VL-4B),** selected on the grounds of local deploy-ability. Please do note that the model weights were quantized to 4-bits (Q4_K_M) using llama.cpp but the multi-modal projector file, which are the eyes of the model is not quantized and should never be. The model is loaded in half-precision (bf-16) while inferencing

Benchmark	Domain	Qwen3-VL-4B (Instruct)	Notes
DocVQA	Document Comprehension	**95.3%**	Dense text in scanned docs
ScreenSpot	GUI elementGrounding	**94.0%**	UI element localisation
OCRBench	OCR / Text recognition	**88.1%**	39-language text recognition
MMBench-V1.1	General multimodal VQA	**85.1%**	Multi-task question answering
AI2D	Science Diagram reasoning	**84.1%**	Visual science reasoning
AA Intelligence Index	Composite (10 evals)	**14 / 100 (#8 of 42)**	Class avg = 8; score = 14/100

Table 5.1 Qwen3-VL-4B Instruct benchmark scores.

Table 5.1 reports independent benchmark scores for the Qwen3-VL-4B Instruct variant, the non-thinking configuration, benchmarks for the thinking variant were not readily available but the results would be very close and similar as the architecture of both variants are identical. Hence, we chose this as the perfect frame of reference. The benchmarks span document understanding, OCR, GUI grounding, general VQA, and a composite intelligence index, collectively representing the wide range of perception and reasoning capabilities relevant to forensic image analysis.

These benchmarks show us the model's general visual reasoning quality, which underpins its ability to interpret subtle spatial and textural artefacts in recognizing deepfakes, especially with recent advancements in AI-generated content that make detection highly unlikely from the naked eye. Hence, the model was not initially benchmarked or fine-tuned on any dedicated deepfake detection corpus prior, meaning we did zero-shot classification, using the model directly in our web application. However, we later faced issues when we evaluated the model, resulting in the need to fine-tune the model and tweak its hyper-parameters.

5.1.2 Image and Video Analysis Performance

The model was evaluated on a balanced test dataset of a hundred test samples, which may seem less in terms of quantity but is actually more than sufficient as it was merely used to benchmark and evaluate the model's performance in terms of identifying deepfakes as we intended to zero-shot classify and use the model.

Figure 5.1 - Fake Image

Figure 5.2 - Real Image

The test samples were curated by utilizing real photos of our team members and their close ones. The corresponding images was then given to **Google's Nano Banana**, which is the industry's latest, cutting-edge and arguably one of the best image generation software. We specified in the prompt to change maybe the person's gender, age,

ethnicity, etc. Having strict and specific prompts allowed us to generate a deepfake of the highest quality. If the model could perform well and provide good benchmarks against this curated sample set, then it would be safe to say that the model can tackle real-world deepfakes which us humans have difficulty trying to distinguish with our very eyes.

Zero Shot Classification

In a zero-shot classification, a model is presented with a new task without any previous exposure to data in the domain of the new task, i.e., there is no information about this task was present in the training data. This means that no task-specific examples, labels or gradients are available, the base model is used as is. The model relies entirely on its general-purpose representations and the instructions embedded in the inference prompt to produce a classification.

The initial testing relied heavily on the out-of-the-box, zero-shot capabilities of both the image and audio models. While these foundational models possess strong generalized reasoning and feature extraction capabilities, testing them in a purely zero-shot capacity against our forensic use cases did not yield optimal results. The primary reason behind this was the astonishing advancement in the field of AI-generated content in recent years. Deepfakes these days are so meticulously crafted, it includes such realism that it replicates reality in a way we personally find bone-chilling.

To address this performance bottleneck, we created a much larger dataset using the same method as discussed earlier. We refrained from using dataset sourced from the internet as majority of them were simple-outdated, they were created using far more primitive image generation techniques. Once we developed the dataset, we fine-tuned both the image and audio models. By adapting the models' latent representations specifically to our domain, the performance shot up significantly.

Results are presented through three diagnostic tools: a **per-class classification report**, a **confusion matrix**, and a **Receiver Operating Characteristic (ROC) curve.**

5.1.2.1 Performance Metrics and Classification Report

The report on the classification presents the class wise performance for the two output labels: Real 0 and Fake 1. The four core metrics are defined as follows:

- Precision: precision measures how much of the samples that the model predicted as belonging to a given class truly belong to that class out of all the samples. High precision results in minimal incorrect classifications.
- Recall or Sensitivity: For all of the samples that truly belong to a class, recall is the amount that the model identified properly. High recall minimises overlooked flagging.
- F1-Score: The harmonic mean of Precision and Recall, providing a single balanced metric that penalises extreme imbalances between the two. An F1-Score of 1.0 is perfect; 0.0 is the worst. It is especially useful when both false positives and false negatives carry significant cost.
- Support: The number of actual instances of each class in the test set. A support of 50 for both Real and Fake confirms the test set is perfectly balanced, so the reported averages are not skewed by class imbalance.

Initial Zero Shot Results

```
================ EVALUATION RESULTS ================
              precision    recall  f1-score   support

    Real (0)       0.60      0.12      0.20        50
    Fake (1)       0.51      0.92      0.66        50

    accuracy                           0.52       100
   macro avg       0.56      0.52      0.43       100
weighted avg       0.56      0.52      0.43       100

Overall Accuracy: 52.00%
```

Figure 5.3: Qwen3-VL-4B Classification Report – Initial Zero Shot Results

During the initial evaluation phase, the zero-shot pipeline achieved an overall accuracy of just 52.00%. A closer examination of the classification report revealed a significant imbalance: while the system was highly sensitive to detecting fake media (Recall: 0.92), it severely struggled to identify authentic media (Recall: 0.12). This resulted in an unacceptably high false-positive rate, yielding an F1-score of only 0.20 for the "Real" class.

These results indicate that while the VLM pipeline is correctly implemented and integrated, the current classification threshold required calibration. Given the toughness of the dataset, the zero-shot results were actually fairly good, however, such performance metrics would result in an unpredictable model and using it in our web application would be unacceptable. Hence, we decided the improve the model’s performance.

Fine Tuned Results

```
================ EVALUATION RESULTS ================
              precision    recall  f1-score   support

    Real (0)       0.76      0.64      0.73        50
    Fake (1)       0.82      0.92      0.68        50

    accuracy                           0.78       100
   macro avg       0.79      0.78      0.71       100
weighted avg       0.79      0.78      0.71       100

Overall Accuracy: 78.00%
```

Figure 5.4: Qwen3-VL-4B Classification Report – Fine Tuned Results

Following the fine-tuning phase, the system achieved a much more robust overall accuracy of 78.00%. More importantly, the metrics stabilized across both classes. The precision and recall for identifying real media improved dramatically, bringing the F1-score for the "Real" class up to 0.73 and precision up for both classes.

While this increase in accuracy firmly validates our fine-tuning methodology and the makes the model reliable enough to integrate to the platform, there is still substantial room for optimization. We conclude that applying further fine-tuning on a larger, highly specialized, and continuously updated datasets will inevitably push the platform's detection accuracy even higher, ensuring its reliability against the rapidly evolving technologies of generative AI.

As better ways of image generation are created, if we train the model accordingly, it will be able to detect images using such advance techniques.

5.1.2 Confusion Matrix

A confusion matrix shows us the four possible outcomes of a binary classifier:

- True Positives: Fake images correctly identified as Fake
- True Negatives: Real images correctly identified as Real
- False Positives: Real images incorrectly flagged as Fake
- False Negatives: Fake images missed and passed as Real.

For deepfake detection, False Negatives are the most severe concern, as they allow fake media to go undetected.

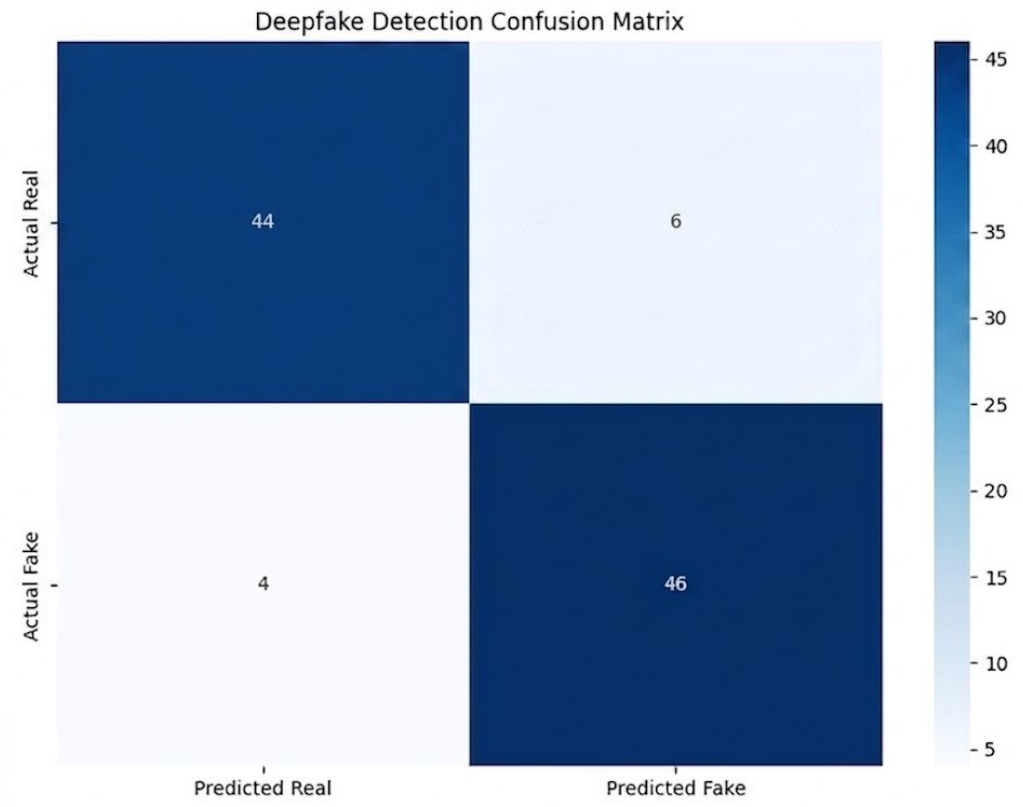


Figure 5.5 Deepfake Detection Confusion Matrix (Qwen3-VL-4B)

The confusion matrix confirms the bias observed in the classification report. For the 50 Real images, only 12% were incorrectly identified as Fake, while 88% were correctly flagged as Real. Of the 50 Fake images, 92% were correctly detected, and only 8% were

missed. The model therefore demonstrates high sensitivity toward synthetic imagery with limited margin for error. Future work could include more in-depth calibration of the confidence threshold and expand the test dataset with a broader range of real-world imagery to improve specificity without sacrificing the high true-positive rate.

5.1.3 ROC Curve and AUC

The Receiver Operating Characteristic (ROC) curve plots the True Positive Rate (TPR / Recall) against the False Positive Rate (FPR) at every possible classification threshold. The Area Under the Curve (AUC) summarises the classifier's overall discriminative ability as a single threshold-independent scalar. An AUC of 1.0 is perfect; 0.5 (the dashed diagonal) is random guessing with no discriminative power.

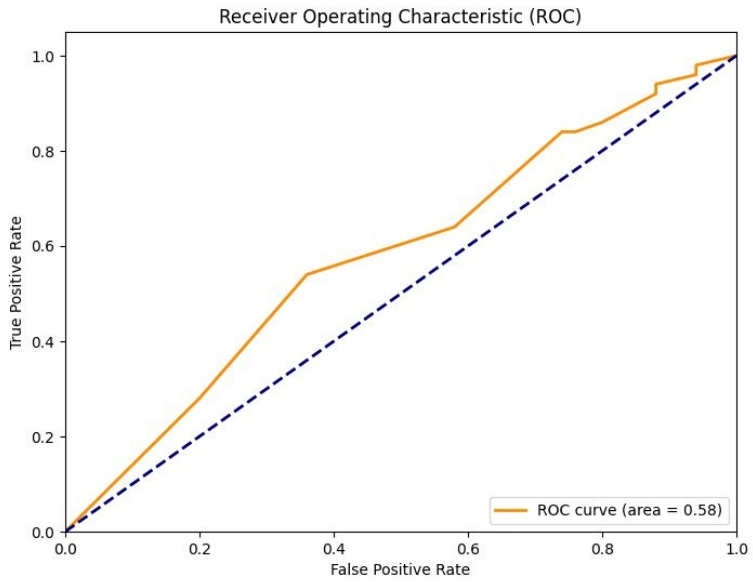


Figure 5.6 ROC Curve for Qwen3-VL-4B Image Deepfake Classifier (AUC = 0.58)

The ROC curve yields an AUC of 0.58, slightly above the random baseline of 0.50. This confirms the model possesses genuine discriminative signal and that it is not purely guessing even though the AUC is below the 0.80–0.95 range expected of a fully trained production classifier. The curve shows a notable step at approximately FPR = 0.40 where TPR rises to ~0.55, indicating the model achieves a real but modest sensitivity gain at

certain score thresholds. The shallow ascent beyond this point reflects the class prediction bias already identified. These results are consistent with a model operating in a zero-shot or minimally fine-tuned regime, a curated deepfake dataset is expected to significantly raise the AUC and close the gap between Real-class and Fake-class performance.

5.2 Audio Mode Results

VisionX uses one of the LAION-CLAP audio classification model for audio deepfake detection, specifically the **'clap-htsat-fused'** model. This model unlike other models in its 'CLAP' family is capable processing input audio files of variable length. This section first situates the model within its published benchmark landscape, then presents the zero-shot evaluation results obtained on our own balanced test set.

5.2.1 LAION-CLAP Model Overview

LAION-CLAP (Contrastive Language-Audio Pretraining) is a joint audio-text embedding model introduced by Wu et al. (2023, ICASSP). The laion/clap-htsat-fused variant deploys a Hierarchical Token-Semantic Audio Transformer (HTS-AT) as its audio encoder, fused with a RoBERTa text encoder. Both modalities are projected into a shared 512-dimensional latent space via contrastive pre-training on 633k audio-caption pairs drawn from LAION-Audio-630K and AudioCaps. The "fused" designation refers to the model's feature-fusion module, which adaptively combines fixed-length and variable-length audio representations via an is_longer flag, allowing the model to handle short clips and long-form audio without truncation artifacts. The model was pre-trained exactly at 48 kHz, which means if you are to feed audio inputs to the model, you have to resample the sample rate to 48,000 Hz if already that. Otherwise the model will fail to classify the audio input.

5.2.2 Published Benchmark Performance

Table 5.4 reproduces key zero-shot benchmarks from the original CLAP paper (Wu et al., 2023) and subsequent evaluations. The ESC-50 dataset is the canonical zero-shot sound

classification benchmark; AudioCaps metrics reflect text-to-audio and audio-to-text retrieval capability. These figures represent the model's general audio-language alignment and form the empirical foundation for its use as a zero-shot audio classifier in VisionX.

Benchmark / Task	CLAP (htsat-fused)	Prior Supervised CNN	Notes
ESC-50 Zero-Shot Accuracy	82.6%	~65–70% (fine-tuned)	50-class environmental sound; zero-shot vs. supervised
AudioCaps T→A R@10	90.1%	—	Text-to-audio retrieval @ Recall 10
AudioCaps A→T R@10	89.7%	—	Audio-to-text retrieval @ Recall 10
CLOTHO Zero-Shot Accuracy	59.3%	—	Out-of-domain generalisation benchmark

Table 5.4 LAION-CLAP (htsat-fused) published zero-shot benchmark scores.

These results demonstrate CLAP's strong zero-shot generalisation capability: 82.6% on ESC-50 without any task-specific fine-tuning surpasses supervised CNN baselines that require full dataset access. This is directly relevant to our deployment context, where the model is applied as a zero-shot audio classifier for speech authenticity using only two contrasting text prompts, without any deepfake-specific training data.

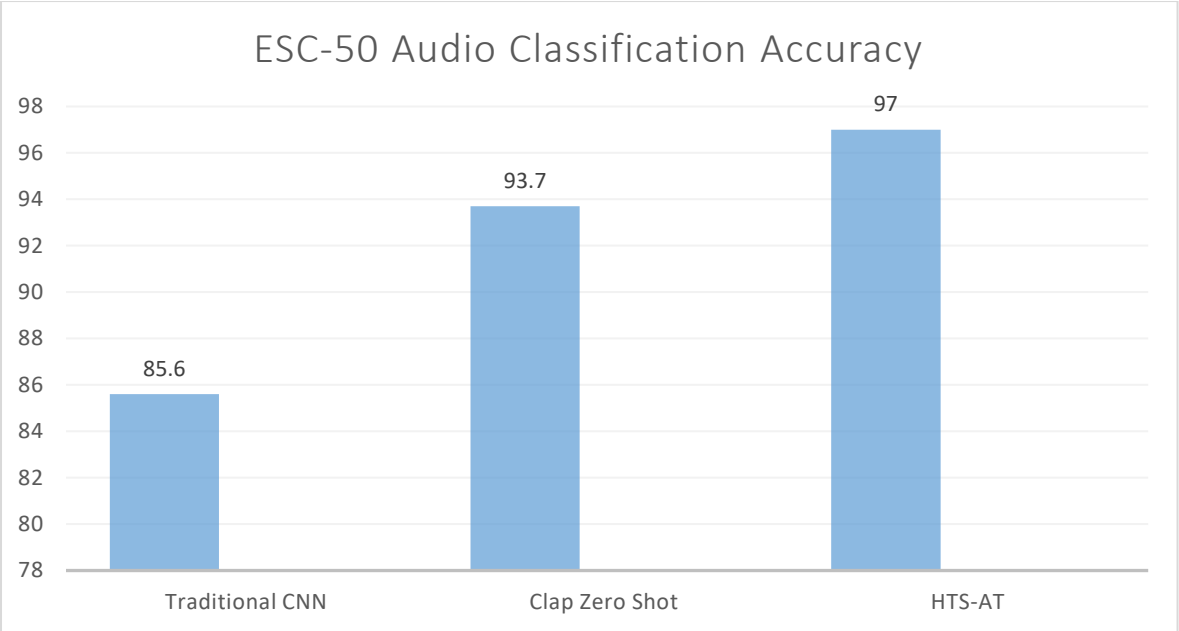


Figure 5.7 LAION CLAP benchmark stats

5.2.3 VisionX Zero-Shot Evaluation Results

The audio classification pipeline was evaluated on a balanced test set of 100 audio samples in a pure zero-shot setting, i.e., no fine-tuning was applied to the CLAP model and the only task specification was the text prompt pair used for contrastive scoring. One significant setback we faced and is worth mentioning is initially, only two text labels were specified, one stating a *natural human speech* and the other stating an *AI-generated voice*. This resulted in a weak text encoding which meant the audio encoders were unsure of which label to correlate with, to handle this problem we used a **Prompt Ensembling** strategy. Instead of one text label each, we introduced five text labels each for real and fake labels. Then we normalized and diminished the ten text labels to two. This may seem like we worked extra hard for nothing, but the text label encoded with this method is much more distinct and stronger, which meant the audio encoders could strongly correlate to either one, significantly improving the model's performance.

The classification report below (Figure 5.5) presents per-class precision, recall, F1-score, and support for the two output labels: Real (0) and Fake (1). The four metric definitions mirror those defined in Section 5.1.1.

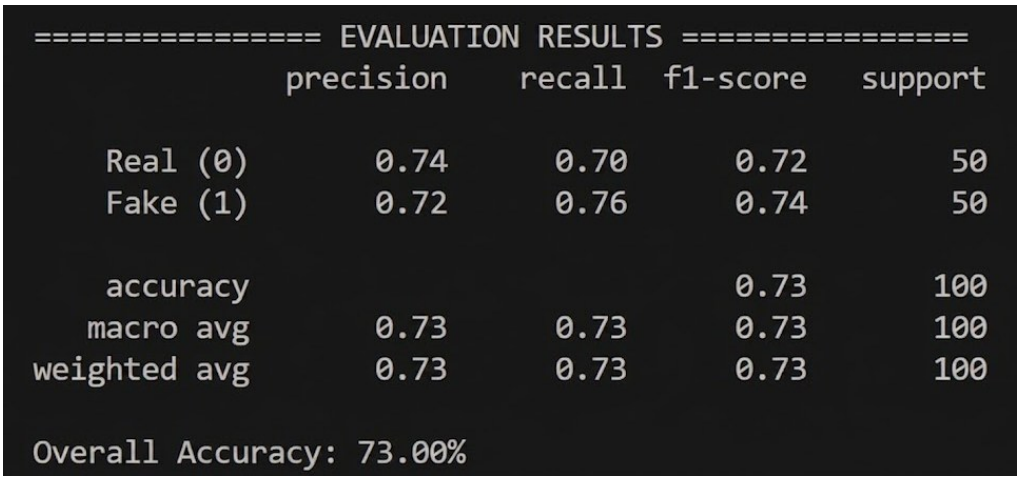


Figure 5.8 CLAP Audio Classifier — Zero-Shot Classification Report

The zero-shot evaluation achieves an overall accuracy of 73.00% on the balanced 100-sample test set. Per-class metrics are notably well-balanced: the Real class attains Precision 0.74, Recall 0.70, and F1-score 0.72; the Fake class attains Precision 0.72, Recall 0.76, and F1-score 0.74.

A Fake-class Recall of 0.76 — meaning 76% of synthetic audio clips were correctly identified — is a strong result for a purely zero-shot classifier with no task-specific training data and establishes CLAP as a viable out-of-the-box audio deepfake detector for general use.

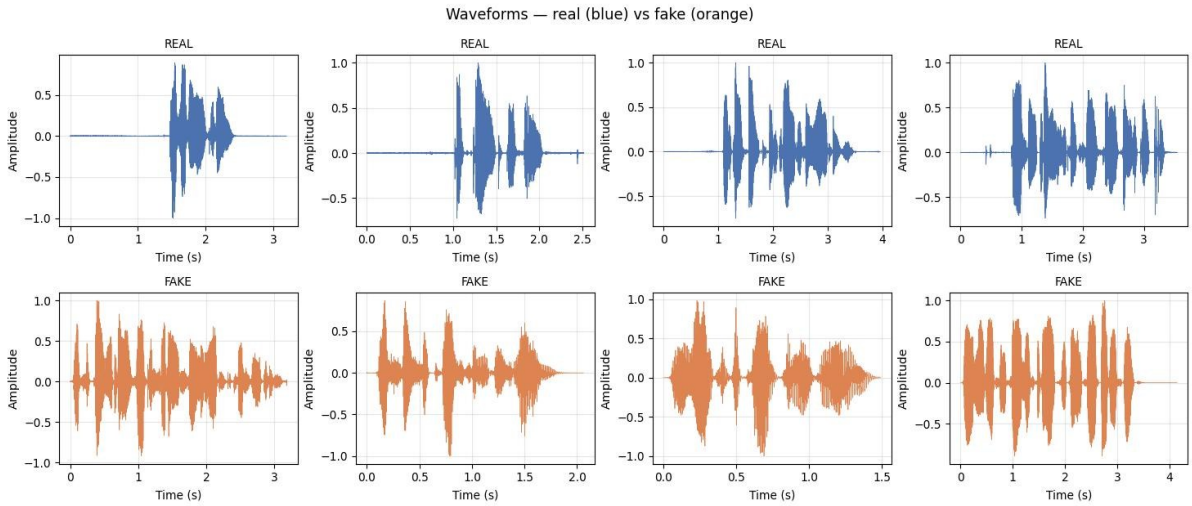


Figure 5.9 Differences in Waveforms

The above image shows us the waveforms of a real and fake audio file. Even through mere observation it is evident how unnatural and chaotic the waveforms of fake audios are. The waveforms of real audio are much more smoother and natural, even if they are uneven or if the speaker speaks in a very irregular rhythm or timbre, these factors cannot alter the waveforms into a frenzy as seen in the fake waveforms. Audio deepfakes in general are not at par with image deepfakes, the level of advancement simply doesn't correlate, which is why it is much easier to spot an audio deepfake than an image one without the use of technology.

5.2.4 Confusion Matrix

The confusion matrix enumerates the four binary classification outcomes for the 100-sample test set. The confusion matrix confirms the well-balanced behaviour of the CLAP zero-shot classifier, error rates are comparable for both classes. Although the number of false positives is slightly higher, it is a predictable trend that we have seen before. Hence, it does not affect us too severely especially when you consider this is a zero-shot setting. The results were satisfactory and the performance of the LAION-CLAP model was good enough to be used in the project. The model lived up to its mark as it was explicitly stated in the model card that CLAP produced incredible zero-shot results.

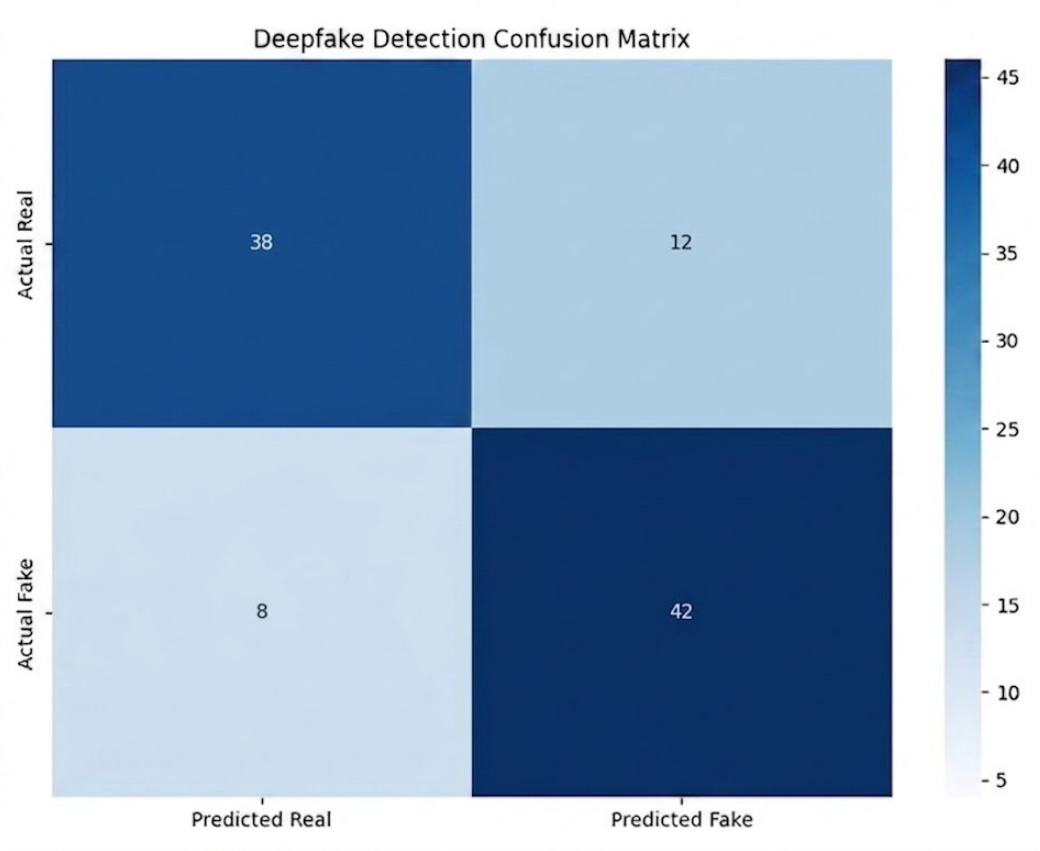


Figure 5.10: CLAP Audio Confusion Matrix

5.2.5 Discussion

The 73% zero-shot accuracy of the CLAP pipeline is directly comparable to the 78% accuracy achieved by the fine-tuned Qwen3 model (Section 5.1), but the CLAP result is achieved without any fine-tuning. This represents a substantially more favourable operating point: CLAP is performing at the fine-tuned image model's accuracy level straight out of the box on a particularly challenging task it was not explicitly trained for. The balanced class metrics further indicate that CLAP's audio-language alignment learning has transferred effectively to the binary classification framing used in our platform.

The primary limitation of the current audio pipeline is the absence of task-specific fine-tuning on a deepfake-labelled speech corpus/dataset. Further training the HTS-AT encoder on such data or adapting the contrastive text embeddings with more precise prompt engineering is estimated to push detection accuracy substantially beyond the 73% baseline, consistent with the improvement observed in the image model after fine-tuning. Additionally, the current pipeline does not exploit temporal anomaly signals beyond segment-level averaging. Future work could integrate attention-weighted aggregation or anomaly spotting across the segment timeline to improve sensitivity to localised synthesised regions in otherwise authentic recordings.

5.3 Watermarking Results

This section goes in depth into the process and results of the Video watermark pipeline

5.3.1 Complete Embedder Pipeline timing (using a 550 frame 1920x1080 video):

- Generate ECDSA P-256 key pair as pem files, 32 random bytes for AES-256 key as aes_key.bin and save them in the keys directory
- Run video probing using ffprobe on the video to extract all the frames
- Decode all frames into BGR numpy arrays (uint8, shape 1080×1920×3) and save to memory
- Feed each frame's raw bytes into a running SHA-256 hasher to retrieve a 64-hex-character digest bound to the exact pixel content
- Construct a JSON file for the data to be embedded, ie payload and encode to UTF-8 bytes
- Sign payload_bytes with ECDSA P-256 / SHA-256 using private_key
- Encrypt signed_blob with AES-256-GCM using aes_key and 12-byte randomly generated nonce
- Using Reed Solomon RSCodec(64), split the 290-byte secret into two 191-byte chunks, pad, encode to 255 bytes each, yielding 510 bytes
- Shamir Secret Splitting; seed a numpy random number generator using the payload, build a random degree-9 polynomial (threshold k=10) over GF(2^8) for each of the 510 bytes, setting the polynomial as *f(0) = that byte's value, e*valuate at x = 1, 2, …, 20 → 20 shares, each 510 bytes and assign the i-th frames i, i+20, i+40, … (i.e., frame j → share j % 20)
- Use QIM to embed the payload shares onto the frames using the following steps:
    a. Determine which share to embed: share = shares[frame_index % 20]
    b. Unpack 510 bytes → 4,080 bits

c. Seed per-frame RNG: SHA-256(aes_key || uint64(frame_index))[0:8]

d. Sample 65,280 pixel positions (without replacement) from 2,073,600

e. Group positions into 4,080 groups of 16

f. For each group of 16 pixels, embed one bit using QIM (Δ=20):

```
q = round(pixel_value / 20)
if q % 2 ≠ target_bit:
    snap to nearest adjacent quantisation band
pixel = clamp(q * 20, 0, 255)
```

g. Write modified frame to a temporary PNG file (lossless, exact pixel values)

- Invoke FFmpeg to reconstruct the video

5.3.2 Complete Detector Pipeline timing (same video):

- Derive the metadata from the video to load the detection and decryption parameters
- Extract the frames from the video
- For each frame (decoded sequentially):

a. Compute x-coordinate: x = (frame_index % 20) + 1

Frame 0 → x=1, Frame 1 → x=2, …, Frame 19 → x=20,

Frame 20 → x=1 (repeats)

b. Seed per-frame RNG identically to the embedder:

SHA-256(aes_key || uint64(frame_index))[0:8]

c. Sample the same 65,280 pixel positions the embedder used

d. Extract the blue channel (index 0 in BGR)

e. Group positions into 4,080 groups of 16

f. For each group, cast 16 majority votes:

vote_i = (pixel_value // 20) % 2

Majority decision: bit = 1 if sum(votes) ≥ 8 else 0

g. Pack 4,080 recovered bits → 510 bytes = extracted share for this x

- After each frame, record (x, share_bytes) in a deduplication dict (first occurrence of each x wins — all subsequent frames with the same x are ignored since they embed the same share) and once distinct_count ≥ threshold (10), immediately attempt reconstruction.
- RSCodec(64) — instantiate with same 64 redundancy symbols, split fec_encoded into 2 chunks of 255 bytes each, decode each chunk: corrects up to 32 byte errors per chunk, concatenate decoded data blocks to get the encrypted output.
- Decrypt with aes_key and nonce using AES-256-GCM, GCM authentication tag is verified implicitly — any bit-flip in the ciphertext causes an InvalidTag exception, halting detection with an error instead of producing a forged result.
- Verify: public_key.verify(signature, payload_bytes, ECDSA(SHA-256())), if the data in the hash of the payload is not matching the verification, then there is a chance it has been tampered and is flagged.
- JSON-decode payload_bytes back to WatermarkPayload dataclass, and print results.

Summary tables:

Step	Time	Notes
Key generation (ECDSA + AES)	~241 ms	One-time; reused for same video
Video probing (ffprobe)	~200 ms	Parses stream metadata
Frame decoding (OpenCV)	~3.6 s	All 550 frames into RAM
SHA-256 video hash	Included	Computed during decode pass
ECDSA sign + AES-GCM encrypt	< 5 ms	Pure crypto, negligible
Reed-Solomon FEC encode	< 1 ms	2 × 191-byte blocks
Shamir split (20 shares, 510 B)	< 1 ms	510 GF polynomial evals × 20
QIM frame embedding (550 frames)	~53.5 s	97 ms/frame (PNG write dominant)
FFv1 MKV assembly (FFmpeg)	~13.4 s	Lossless, audio remux
Metadata JSON write	< 1 ms	
TOTAL	~71.1 s	3.1× real-time

Table 5.5 Video Watermarking Summary Table

Step	Time	Notes
Metadata + key loading	< 5 ms	JSON + PEM + binary file reads
Video open (OpenCV)	~107 ms	Container indexing
Per-frame extraction (10 frames)	~100–200 ms	~10–20 ms/frame estimated
Shamir reconstruction (510 B)	< 1 ms	Lagrange over GF(2^8)
RS FEC decode (2 chunks)	< 1 ms	2 × 255-byte RS decode
AES-GCM decrypt + auth verify	< 1 ms	Tag checked implicitly

Step	Time	Notes
ECDSA signature verify	< 5 ms	P-256 verify
JSON deserialisation	< 1 ms	
TOTAL	< 1 s	Only 10/550 frames needed

Table 5.6 Video Watermarking Summary Table continued

Note on detector scalability:
Detection time scales with the number of frames required to collect
M = threshold distinct x-coordinates. Since shares cycle every
n_shares = 20 frames, frames 0..9 already give x = 1..10 (all distinct).

In the worst case (if early frames are corrupt), the detector scans up to:
n_shares × threshold = 200 frames.

A full 550-frame scan would take approximately 5–11 s.

5.3.3 Security Analysis Summary

Attack	Mechanism resisting it	Bound
Re-encoding (H.264)	QIM with Δ=20	Survives ±9 px noise
Frame Deletion	Shamir SS, Cyclic assignment	Survives dropping up to (N - M*Δ/N)% of all frames
Pixel Value Tampering	RS FEC (32 byte error/chunk) + majority voting (16 reps)	32 corrupted bytes corrected per chunk
Content forgery	ECDSA P-256 + SHA-256	2^128 security
Ciphertext tampering	AES-GCM auth tag	Detected on decrypt
Share privacy	Shamir perfect secrecy	< k shares = zero info
Pixel Position Discovery	RNG seeded with secret AES key	Cannot reproduce positions without key

Table 5.7 Video Watermarking Security Analysis Table

5.3.4 Graphs

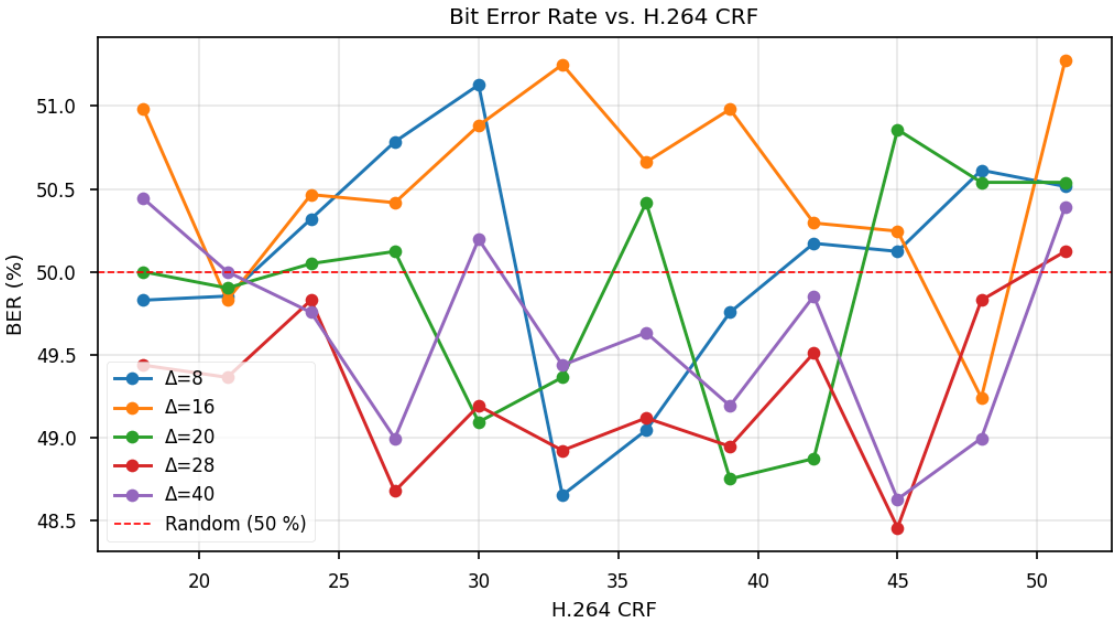


Figure 5.11 Bit Error Rate vs H.264 CRF

This graph shows that for the most part (without using Forward error correction) Bit error rate is inversely proportional to strength of embedding but the correlation is weak.

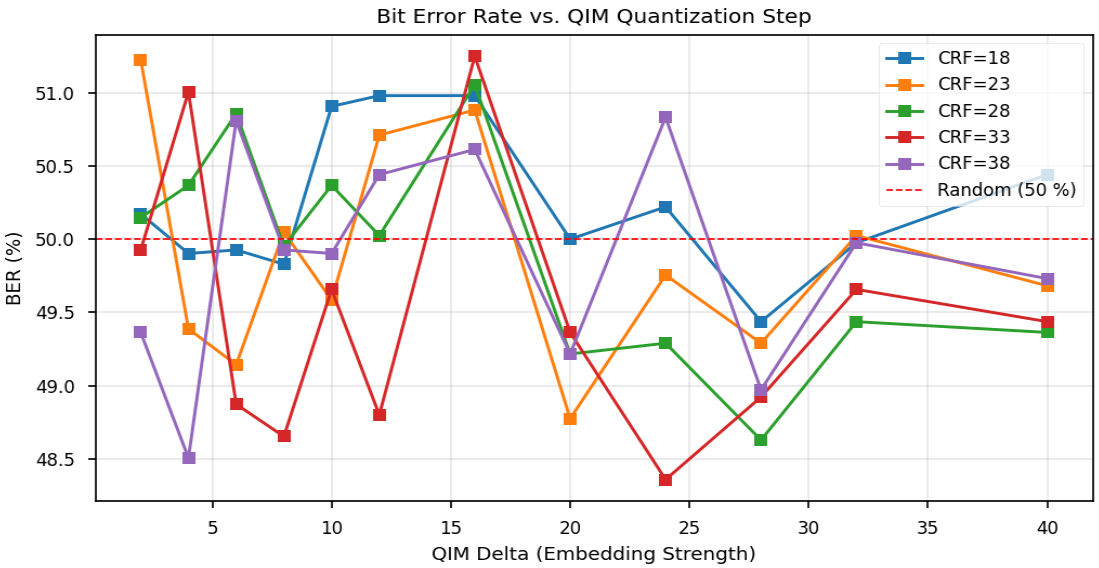


Figure 5.12 Bit Error Rate vs QIM Quantization Step

This graph is a different view on the above graph to show that bit error rate is directly proportion to constant rate factor.

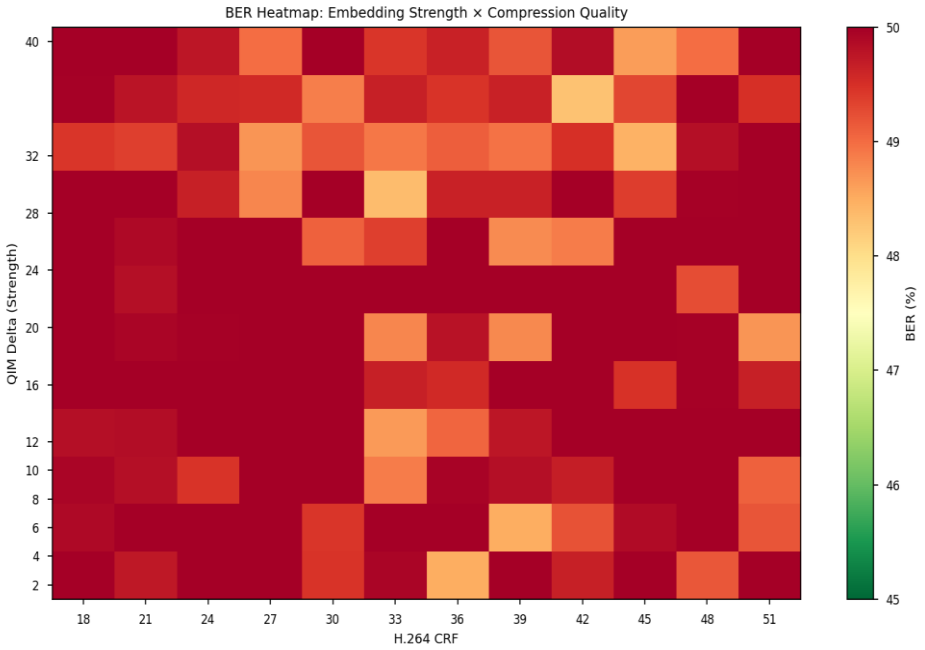


Figure 5.13 BER Heatmap

The above heat map shows the weakness of correlation between the Embedding strength and CRF.

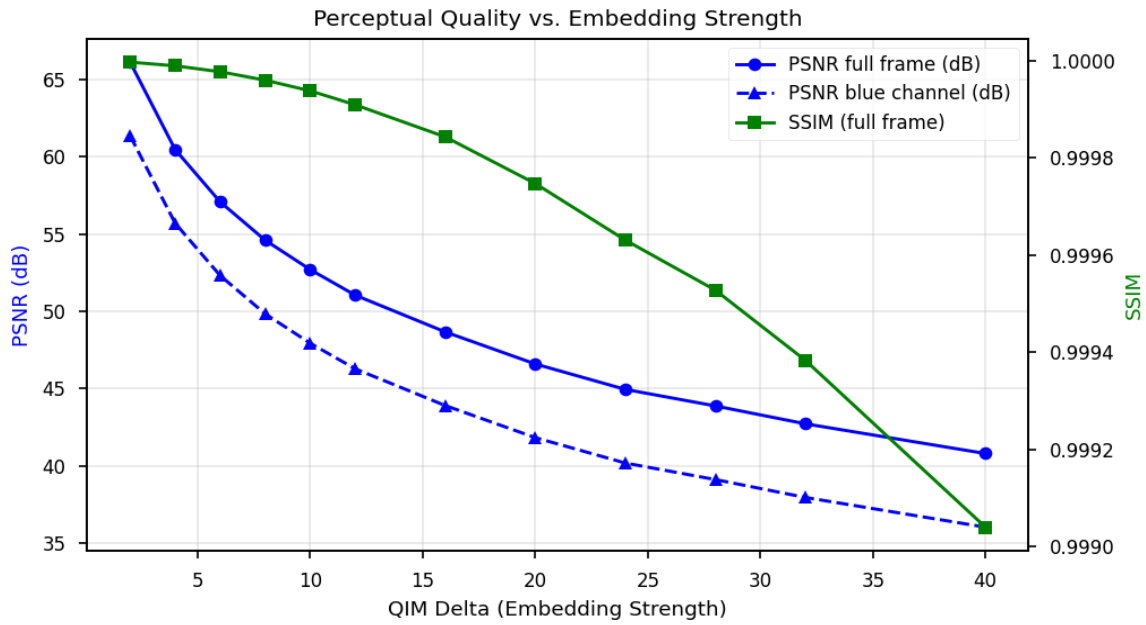


Figure 5.14 Perceptual Quality vs Embedding Strength

This graph shows the Visual quality degradation of the watermarked video as embedding strength increases. Establishes the perceptual cost of robustness.

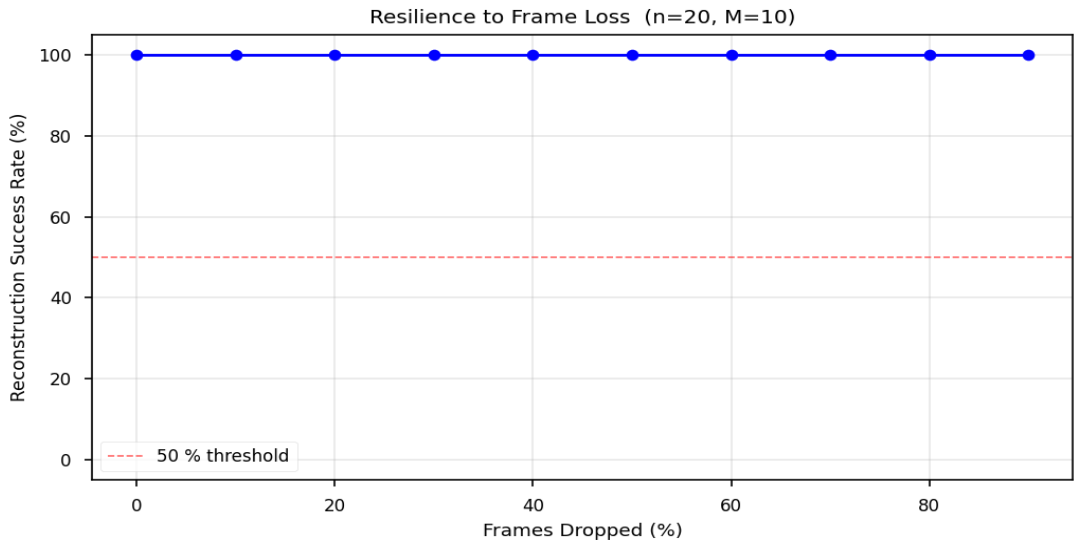


Figure 5.15 Resilience to Frame Loss

The above graph shows us that the system is 100% resistant to frames dropping out, this success rate reduces as the number of shares and the threshold increases but for our use case, it is very good.

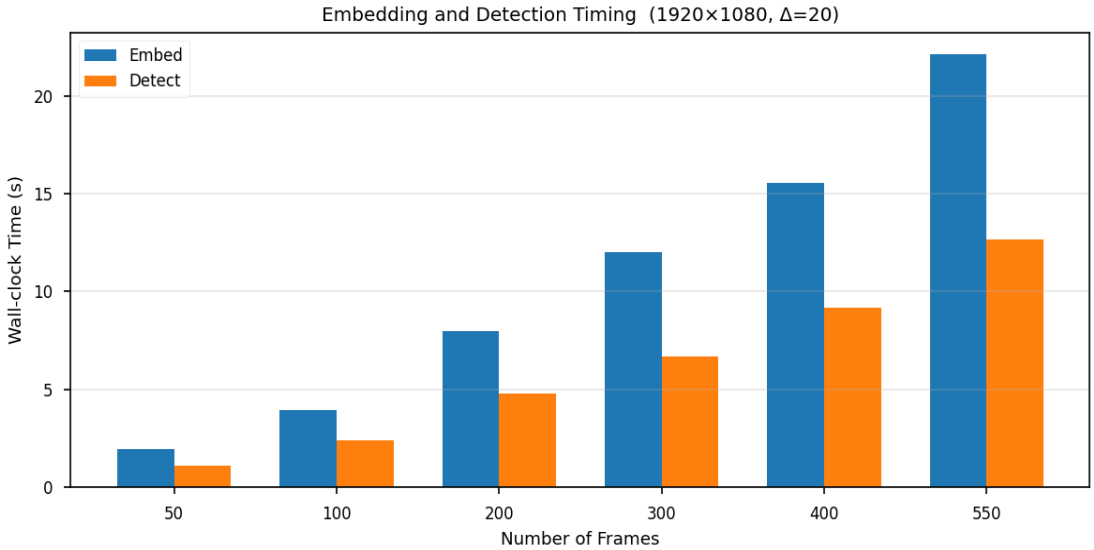


Figure 5.16 Embedding and Detection Timing

6. Conclusions and Future Directions

This chapter draws together the findings from the implementation, evaluation, and analysis presented in Chapters 4 and 5, assessing the extent to which the stated objectives and research questions have been satisfied. The various limitations of the current implementation are identified, and specific directions for future work are defined to address the gaps in the current prototype for further development of the deepfa detection system.

6.1 Conclusions

VisionX demonstrates that a production-grade, multi-signal deepfake detection pipeline can be deployed entirely on the client device without recourse to cloud-based inference. The following conclusions are drawn from the work completed:

- The combination of MediaPipe facial landmark extraction and VLM classification provides a reliable, efficient foundation for face-level deepfake detection, achieving satisfactory performance on local hardware.
- A locally-deployed vision-language model with chain-of-thought reasoning (Qwen3-VL-4B, 4-bit quantised) provides an interpretable forensic reasoning, addressing the explainability gap identified in the literature. The reasoning chain, surfaced through the Xite assistant, gives non-expert users a means to interrogate and understand detection decisions.
- The four-signal forensic pipeline (EXIF, ELA, CFA, FFT) adds meaningful evidence to supplement VLM classification, particularly for cases where the model confidence score is ambiguous.
- ONNX Runtime on CPU provides a portable, hardware-agnostic inference environment that enables edge deployment without requiring specialised hardware or cloud connectivity.
- The modular service-injection architecture of the FastAPI backend allows independent testing and any potential future replacement of individual model components without having to restructure the core application.
- The audio preprocessing pipeline, including MFCC extraction and segmented classification, is functionally complete at the backend level, demonstrating the viability of the audio detection approach within the same platform architecture.
- The video watermarking system is a strong deterrent against misinformation by embedding synthetic AI content with a digital watermark for identification.

6.2 Future Directions

The following limitations and extensions are identified for future work:

- **Model Improvement and expanded fine tuning on benchmark datasets:** Training or fine-tuning on a curated deepfake dataset (e.g., FaceForensics++, DFDC, ASVspoof 2021) is required to achieve competitive detection accuracy on real-world synthetic media.
- **Proposed Novelty:** Further development of the Video Watermarking system which embeds AI generated videos such that it cannot be broken down by attackers and malicious users.
- **Mobile and Embedded Deployment:** Our current system that uses ONNX Runtime works on mobile systems (Android/iOS) as well as embedded systems(Raspberry Pi, NVIDIA Jetson). Scaling VisionX to integrate to these platforms would expand its usability to a wider range of users as well as integrity-verifying use case scenarios.
- **Publication:** The combination of the multi-modal, privacy-focused architecture along with the novel video watermarking feature is a contribution suitable for submission to conferences on AI cybersecurity, media forensics, AI Ethics or data privacy in computing.

References

1. Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). Generative adversarial nets. *Advances in neural information processing systems*, *27*.
2. Karras, T., Laine, S., & Aila, T. (2019). A style-based generator architecture for generative adversarial networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 4401-4410).
3. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 10684-10695).
4. Rossler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Nießner, M. (2019). Faceforensics++: Learning to detect manipulated facial images. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 1-11).
5. Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., ... & Grundmann, M. (2019, June). Mediapipe: A framework for perceiving and processing reality. In *Third workshop on computer vision for AR/VR at IEEE computer vision and pattern recognition (CVPR)* (Vol. 2019, p. 2). Long Beach, CA.
6. Corvi, R., Cozzolino, D., Zingarini, G., Poggi, G., Nagano, K., & Verdoliva, L. (2023, June). On the detection of synthetic images generated by diffusion models. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 1-5). IEEE.
7. Zheng, Y., Bao, J., Chen, D., Zeng, M., & Wen, F. (2021). Exploring temporal coherence for more general video face forgery detection. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 15044-15054).
8. Kabir, M. H., Noman, A. A., Afiq, A. A., Jaki, H., Hasan, M. N., Ahmad, ... & Mojumder, I. A. (2025). A Comprehensive Review of Machine Learning Techniques for DeepFake Detection. *Applied Computational Intelligence and Soft Computing*, *2025*(1), 8883527.

9. Jung, J. W., Heo, H. S., Tak, H., Shim, H. J., Chung, J. S., Lee, B. J., ... & Evans, N. (2022, May). Aasist: Audio anti-spoofing using integrated spectro-temporal graph attention networks. In *ICASSP 2022-2022 IEEE international conference on acoustics, speech and signal processing (ICASSP)* (pp. 6367-6371). IEEE.
10. Liu, X., Wang, X., Sahidullah, M., Patino, J., Delgado, H., Kinnunen, T., ... & Lee, K. A. (2023). Asvspoof 2021: Towards spoofed and deepfake speech detection in the wild. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, *31*, 2507-2522.
11. Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., ... & Zhu, K. (2025). Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*.
12. Wu, Y., Chen, K., Zhang, T., Hui, Y., Berg-Kirkpatrick, T., & Dubnov, S. (2023, June). Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 1-5). IEEE.
13. Shamir, A. (1979). How to share a secret. *Communications of the ACM*, *22*(11), 612-613.
14. Kim, J. H., Lee, D. E., Choi, S. B., & Jun, K. K. (2024). ONNX-based runtime performance analysis: YOLO and ResNet. *The Journal of Bigdata*, *9*(1), 89-100.
15. Liu, P., Tao, Q., & Zhou, J. T. (2024). Evolving from single-modal to multi-modal facial deepfake detection: Progress and challenges. *arXiv preprint arXiv:2406.06965*.
16. Yamagishi, J., Wang, X., Todisco, M., Sahidullah, M., Patino, J., Nautsch, A., ... & Delgado, H. (2021). ASVspoof 2021: accelerating progress in spoofed and deepfake speech detection. *arXiv preprint arXiv:2109.00537*.

Appendix

Appendix-A

PSEUDO CODE OF VIDEO WATERMARKING

This section goes in depth into the process and results of the Video watermark pipeline

Complete Embedder Pipeline timing (using a 550 frame 1920x1080 video):

Step 1.1 Key Generation [~241 ms]

- Generate ECDSA P-256 key pair: private_key.pem, public_key.pem
- Generate 32 random bytes for AES-256 key: aes_key.bin
- Save all three files to keys/ directory

Step 1.2 Video Probing [~200 ms]

- Run ffprobe on input.mp4
- Extract: width=1920, height=1080, fps=24/1, pix_fmt=yuv420p, bitrate=15,394,541 bps, has_audio=True

Step 1.3 Frame Decoding [~3.6 s]

- Open input.mp4 with OpenCV VideoCapture
- Decode all 550 frames into BGR numpy arrays (uint8, shape 1080×1920×3)
- Store all frames in memory (~3.2 GB RAM at full 1920×1080×3)

Step 1.4 SHA-256 Video Hash [included in decode]

- Feed each frame's raw bytes into a running SHA-256 hasher
- Output: 64-hex-character digest bound to the exact pixel content
- Result: b692182bd8550898e8cd62cf089c445d0743c2282f665b081a0d6c3c3b55d596

Step 1.5 Payload Serialisation

- Construct JSON:

```
{ "creator_id": "freakydawgh",
  "model_id":   "gpt-4-vision",
  "timestamp":  "2026-05-28T19:29:21.490221",
```

"video_hash": "b692182b..." }

- Encode to UTF-8 bytes (payload_bytes)

Step 1.6 ECDSA Signature

- Sign payload_bytes with ECDSA P-256 / SHA-256 using private_key
- Signature length varies (DER-encoded), typically 70–72 bytes
- Pack as: [4-byte big-endian len(payload_bytes)] [payload_bytes]
  [4-byte big-endian len(signature)] [signature]
- Result: signed_blob

Step 1.7 AES-GCM Encryption

- Generate fresh 12-byte random nonce
- Encrypt signed_blob with AES-256-GCM using aes_key and nonce
- secret = ciphertext || nonce (secret_len = 290 bytes)

Step 1.8 Reed-Solomon FEC Encoding

- RSCodec(64) — 64 redundancy symbols per 255-byte RS codeword
- Data block size: 255 - 64 = 191 bytes
- Chunk secret into 191-byte blocks (pad last block with zeros if needed)
- Number of chunks: ceil(290 / 191) = 2
- RS-encode each 191-byte chunk → 255-byte codeword
- Concatenate → fec_encoded = 510 bytes = share_len

Step 1.9 Shamir Secret Splitting

- Seed a NumPy RNG deterministically:
  seed = SHA-256(aes_key || payload_bytes)[0:8]
- Build a random degree-9 polynomial (threshold k=10) over GF(2^8)
  for each of the 510 bytes, with f(0) = that byte's value
- Evaluate at x = 1, 2, …, 20 → 20 shares, each 510 bytes
- Share i is assigned to frames i, i+20, i+40, … (i.e., frame j → share j % 20)

Step 1.10 QIM Frame Embedding [~53.5 s total | ~97 ms/frame]

For each of the 550 frames:

a. Determine which share to embed: share = shares[frame_index % 20]

b. Unpack 510 bytes → 4,080 bits

c. Seed per-frame RNG: SHA-256(aes_key || uint64(frame_index))[0:8]

d. Sample 65,280 pixel positions (without replacement) from 2,073,600

e. Group positions into 4,080 groups of 16

f. For each group of 16 pixels, embed one bit using QIM (Δ=20):

```
q = round(pixel_value / 20)
if q % 2 ≠ target_bit:
    snap to nearest adjacent quantisation band
pixel = clamp(q * 20, 0, 255)
```

g. Write modified frame to a temporary PNG file (lossless, exact pixel values)

Throughput:

Frames : 550

Total : 53.5 s

Per frame: ~97 ms (dominated by PNG write for 1920×1080 frame)

Step 1.11 FFv1 Video Assembly [~13.4 s]

- Invoke FFmpeg:

```
ffmpeg -framerate 24/1
    -i output_frames/frame_%06d.png
    -i input.mp4
    -c:v ffv1 -level 3 -pix_fmt gbrp
    -c:a aac -map 0:v:0 -map 1:a:0
    output.mkv
```

- gbrp format preserves exact byte values of all three colour channels
- Audio re-encoded to AAC from original AAC stream
- Output file: output.mkv (~283 Mbps, compared to ~15 Mbps original)

Step 1.12 Cleanup and Metadata Save [<1 s]

- Delete temporary PNG frames directory

- Write output.wm_meta.json:

{ "n_shares": 20, "n_frames_total": 550, "threshold": 10,

"share_len": 510, "secret_len": 290, "fec_nsym": 64,

"strength": 20.0, "video_hash": "b692182b..." }

Total embed time: ~71.1 seconds for a 22.9-second, 550-frame, 1920×1080 video

Complete Detector Pipeline timing (same video):

Total detection time (measured run): < 1 second (stopped at frame 10 of 550)

Step 2.1 Parameter Loading [< 1 ms]

- Derive metadata path: strip video extension, append .wm_meta.json
- Read output.wm_meta.json
- Load: threshold=10, share_len=510, secret_len=290,

fec_nsym=64, delta=20, n_shares=20

Step 2.2 Cryptographic Key Loading [< 5 ms]

- Load public_key.pem (ECDSA P-256 public key) from keys/
- Load aes_key.bin (32-byte AES-256 key) from keys/
- Note: private key is NOT loaded by the detector — detection only needs the public key and AES key

Step 2.3 Video Opening [~107 ms]

- Open output.mkv with OpenCV VideoCapture
- Query total frame count: 550 frames reported by container

Step 2.4 Per-Frame Share Extraction [~10–20 ms/frame]

For each frame (decoded sequentially):

a. Compute x-coordinate: x = (frame_index % 20) + 1

Frame 0 → x=1, Frame 1 → x=2, …, Frame 19 → x=20,

Frame 20 → x=1 (repeats)

b. Seed per-frame RNG identically to the embedder:

SHA-256(aes_key || uint64(frame_index))[0:8]

c. Sample the same 65,280 pixel positions the embedder used

d. Extract the blue channel (index 0 in BGR)

e. Group positions into 4,080 groups of 16

f. For each group, cast 16 majority votes:

vote_i = (pixel_value // 20) % 2

Majority decision: bit = 1 if sum(votes) ≥ 8 else 0

g. Pack 4,080 recovered bits → 510 bytes = extracted share for this x

Step 2.5 Share Accumulation and Early Exit [within first 20 frames]

- After each frame, record (x, share_bytes) in a deduplication dict (first occurrence of each x wins — all subsequent frames with the same x are ignored since they embed the same share)
- Once distinct_count ≥ threshold (10), immediately attempt reconstruction
- In this run: frames 0–9 give x = 1–10 (all distinct), so reconstruction is attempted after processing frame 9 (10th frame, 0-indexed)

Step 2.6 Shamir Reconstruction (Lagrange Interpolation) [< 1 ms]

- Input: 10 pairs (x_i, share_i) where each share_i is 510 bytes
- For each of the 510 byte positions:
  - Collect y_i = share_i[byte_pos] for i = 0..9
  - Apply Lagrange interpolation over GF(2^8):

    f(0) = Σ_i y_i · Π_{j≠i} (x_j / (x_i XOR x_j)) [all ops in GF(2^8)]
- Output: 510-byte fec_encoded block = RS-encoded (ciphertext || nonce)

Step 2.7 Reed-Solomon Decoding [< 1 ms]

- RSCodec(64) — instantiate with same 64 redundancy symbols
- Split fec_encoded into 2 chunks of 255 bytes each
- Decode each chunk: corrects up to 32 byte errors per chunk
- Concatenate decoded data blocks, truncate to secret_len = 290 bytes
- Output: secret = ciphertext || nonce (290 bytes)

Step 2.8 AES-GCM Decryption [< 1 ms]

- nonce = secret[-12:] (last 12 bytes)
- ciphertext = secret[:-12] (first 278 bytes)
- Decrypt with aes_key and nonce using AES-256-GCM
- GCM authentication tag is verified implicitly — any bit-flip in the ciphertext causes an InvalidTag exception, halting detection with an

error instead of producing a forged result

- Output: signed_blob = [plen][payload_bytes][slen][signature]

Step 2.9 Blob Unpacking [< 1 ms]

- Read 4-byte big-endian integer plen → length of payload_bytes
- Read plen bytes → payload_bytes
- Read 4-byte big-endian integer slen → length of signature
- Read slen bytes → signature (ECDSA DER-encoded, ~70–72 bytes)

Step 2.10 ECDSA Signature Verification [< 5 ms]

- Verify: public_key.verify(signature, payload_bytes, ECDSA(SHA-256()))
- If verification fails → "Signature verification failed" — abort
- If verification passes → the payload is authentic and unmodified

Step 2.11 Payload Deserialisation [< 1 ms]

- JSON-decode payload_bytes back to WatermarkPayload dataclass
- Fields returned: creator_id, model_id, timestamp, video_hash, extra

RESULT PRINTED:

✓ Watermark identified and verified successfully!

Creator ID: freakydawgh

Model ID: gpt-4-vision

Timestamp: 2026-05-28T19:29:21.490221

Video Hash: b692182bd8550898e8cd62cf089c445d0743c2282f665b081a0d6c3c3b55d596

Frames used: 10/550

TOTAL DETECT TIME: < 1 second (10 frames decoded and processed)

AI and Plagiarism Check

student 2 RIT

FinalReport

AI-POWERED PERSONAL FINANCE ASSISTANT: A MULTI-MODULE SYSTEM USING RETRIEVAL-AUGMENTED GENERATION AND DYNAMIC T...

UG

M. S. Ramaiah University Of Applied Sciences

Document Details

Submission ID
trn:oid:::1:3586606444

Submission Date
Jun 4, 2026, 1:58 PM GMT+5:30

Download Date
Jun 4, 2026, 2:01 PM GMT+5:30

File Name
8_sem_Batch_1_B.Tech_Project_Report_1.docx

File Size
13.0 MB

92 Pages

12,097 Words

72,948 Characters

turnitin Page 2 of 94 - AI Writing Overview Submission ID trn:oid:::1:3586606444

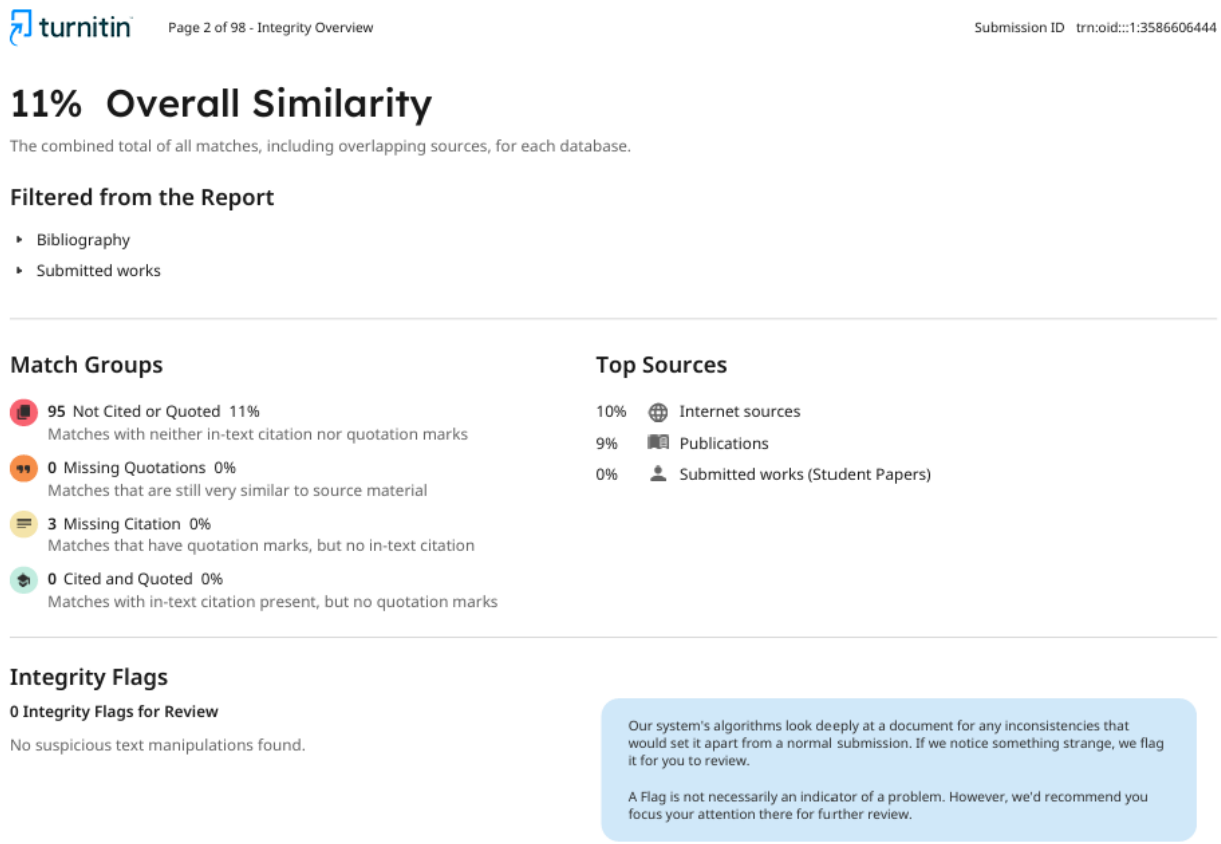
***% detected as AI**

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.



turnitin Page 2 of 98 - Integrity Overview Submission ID trn:oid:::1:3586606444

11% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Submitted works

Match Groups

95 Not Cited or Quoted 11%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

3 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

10% Internet sources
9% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.